13

Reliable Stream Transport Service (TCP)

13.1 Introduction

Previous chapters explore the unreliable connectionless packet delivery service that forms the basis for all internet communication and the IP protocol that defines it. This chapter introduces the second most important and well-known network-level service, reliable stream delivery, and the *Transmission Control Protocol* (*TCP*) that defines it. We will see that TCP adds substantial functionality to the protocols already discussed, but that its implementation is also substantially more complex.

Although TCP is presented here as part of the TCP/IP Internet protocol suite, it is an independent, general purpose protocol that can be adapted for use with other delivery systems. For example, because TCP makes very few assumptions about the underlying network, it is possible to use it over a single network like an Ethernet, as well as over a complex internet. In fact, TCP has been so popular that one of the International Organization for Standardization's open systems protocols, TP-4, has been derived from it.

13.2 The Need For Stream Delivery

At the lowest level, computer communication networks provide unreliable packet delivery. Packets can be lost or destroyed when transmission errors interfere with data, when network hardware fails, or when networks become too heavily loaded to accommodate the load presented. Networks that route packets dynamically can deliver them out of order, deliver them after a substantial delay, or deliver duplicates. Furthermore,

underlying network technologies may dictate an optimal packet size or pose other constraints needed to achieve efficient transfer rates.

At the highest level, application programs often need to send large volumes of data from one computer to another. Using an unreliable connectionless delivery system for large volume transfers becomes tedious and annoying, and it requires programmers to build error detection and recovery into each application program. Because it is difficult to design, understand, or modify software that correctly provides reliability, few application programmers have the necessary technical background. As a consequence, one goal of network protocol research has been to find general purpose solutions to the problems of providing reliable stream delivery, making it possible for experts to build a single instance of stream protocol software that all application programs use. Having a single general purpose protocol helps isolate application programs from the details of networking, and makes it possible to define a uniform interface for the stream transfer service.

13.3 Properties Of The Reliable Delivery Service

The interface between application programs and the TCP/IP reliable delivery service can be characterized by 5 features:

• *Stream Orientation.* When two application programs (user processes) transfer large volumes of data, we think of the data as a *stream* of bits, divided into 8-bit *octets*, which are informally called *bytes*. The stream delivery service on the destination machine passes to the receiver exactly the same sequence of octets that the sender passes to it on the source machine.

• *Virtual Circuit Connection.* Making a stream transfer is analogous to placing a telephone call. Before transfer can start, both the sending and receiving application programs interact with their respective operating systems, informing them of the desire for a stream transfer. Conceptually, one application places a "call" which must be accepted by the other. Protocol software modules in the two operating systems communicate by sending messages across an internet, verifying that the transfer is authorized, and that both sides are ready. Once all details have been settled, the protocol modules inform the application programs that a *connection* has been established and that transfer can begin. During transfer, protocol software on the two machines continue to communicate to verify that data is received correctly. If the communication fails for any reason (e.g., because network hardware along the path between the machines fails), both machines detect the failure and report it to the appropriate application programs. We use the term *virtual circuit* to describe such connections because although application programs view the connection as a dedicated hardware circuit, the reliability is an illusion provided by the stream delivery service.

• *Buffered Transfer.* Application programs send a data stream across the virtual circuit by repeatedly passing data octets to the protocol software. When transferring data, each application uses whatever size pieces it finds convenient, which can be as small as a single octet. At the receiving end, the protocol software delivers octets from

the data stream in exactly the same order they were sent, making them available to the receiving application program as soon as they have been received and verified. The protocol software is free to divide the stream into packets independent of the pieces the application program transfers. To make transfer more efficient and to minimize network traffic, implementations usually collect enough data from a stream to fill a reasonably large datagram before transmitting it across an internet. Thus, even if the application program generates the stream one octet at a time, transfer across an internet may be quite efficient. Similarly, if the application program chooses to generate extremely large blocks of data, the protocol software can choose to divide each block into smaller pieces for transmission.

For those applications where data should be delivered even though it does not fill a buffer, the stream service provides a *push* mechanism that applications use to force a transfer. At the sending side, a push forces protocol software to transfer all data that has been generated without waiting to fill a buffer. When it reaches the receiving side, the push causes TCP to make the data available to the application without delay. The reader should note, however, that the push function only guarantees that all data will be transferred; it does not provide record boundaries. Thus, even when delivery is forced, the protocol software may choose to divide the stream in unexpected ways.

• *Unstructured Stream.* It is important to understand that the TCP/IP stream service does not honor structured data streams. For example, there is no way for a payroll application to have the stream service mark boundaries between employee records, or to identify the contents of the stream as being payroll data. Application programs using the stream service must understand stream content and agree on stream format before they initiate a connection.

• *Full Duplex Connection.* Connections provided by the TCP/IP stream service allow concurrent transfer in both directions. Such connections are called *full duplex.* From the point of view of an application process, a full duplex connection consists of two independent streams flowing in opposite directions, with no apparent interaction. The stream service allows an application process to terminate flow in one direction while data continues to flow in the other direction, making the connection *half duplex.* The advantage of a full duplex connection is that the underlying protocol software can send control information for one stream back to the source in datagrams carrying data in the opposite direction. Such *piggybacking* reduces network traffic.

13.4 Providing Reliability

We have said that the reliable stream delivery service guarantees to deliver a stream of data sent from one machine to another without duplication or data loss. The question arises: "How can protocol software provide reliable transfer if the underlying communication system offers only unreliable packet delivery?" The answer is complicated, but most reliable protocols use a single fundamental technique known as *positive acknowledgement with retransmission.* The technique requires a recipient to communicate with the source, sending back an *acknowledgement* (*ACK*) message as it receives

data. The sender keeps a record of each packet it sends and waits for an acknowledgement before sending the next packet. The sender also starts a timer when it sends a packet and *retransmits* a packet if the timer expires before an acknowledgement arrives.

Figure 13.1 shows how the simplest positive acknowledgement protocol transfers data.

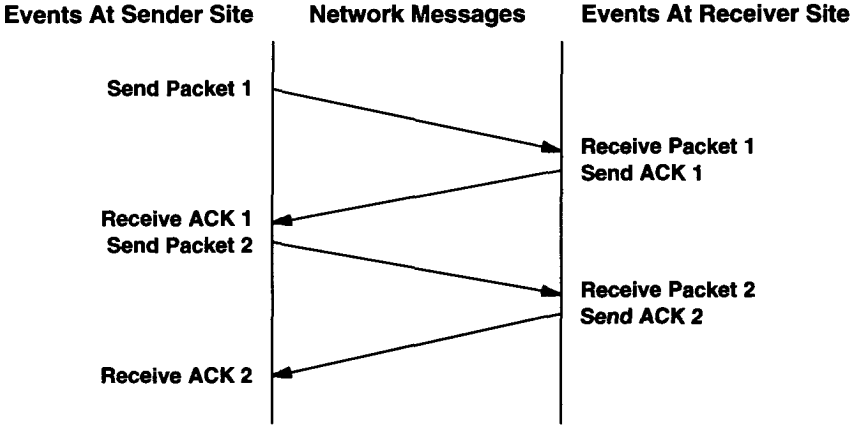

Figure 13.1 A protocol using positive acknowledgement with retransmission in which the sender awaits an acknowledgement for each packet sent. Vertical distance down the figure represents increasing time and diagonal lines across the middle represent network packet transmission.

In the figure, events at the sender and receiver are shown on the left and right. Each diagonal line crossing the middle shows the transfer of one message across the network.

Figure 13.2 uses the same format diagram as Figure 13.1 to show what happens when a packet is lost or corrupted. The sender starts a timer after transmitting a packet. When the timer expires, the sender assumes the packet was lost and retransmits it.

The final reliability problem arises when an underlying packet delivery system duplicates packets. Duplicates can also arise when networks experience high delays that cause premature retransmission. Solving duplication requires careful thought because both packets and acknowledgements can be duplicated. Usually, reliable protocols detect duplicate packets by assigning each packet a sequence number and requiring the receiver to remember which sequence numbers it has received. To avoid confusion caused by delayed or duplicated acknowledgements, positive acknowledgement protocols send sequence numbers back in acknowledgements, so the receiver can correctly associate acknowledgements with packets.

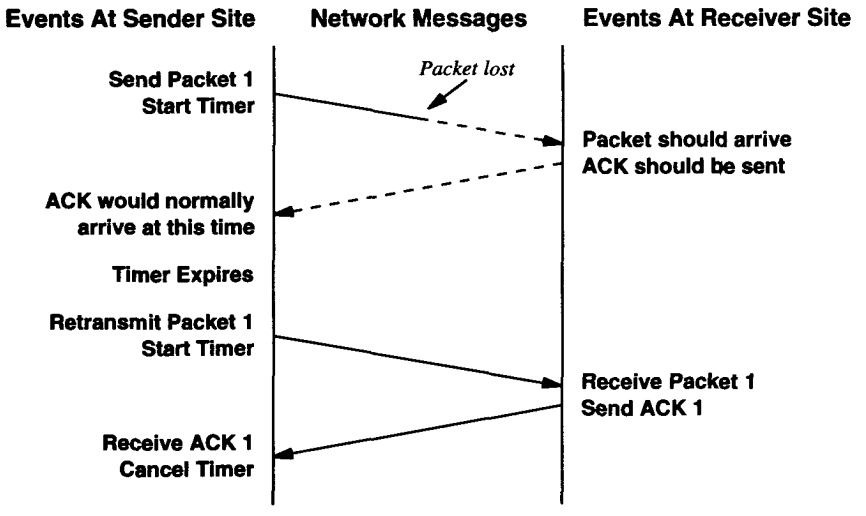

Figure 13.2 Timeout and retransmission that occurs when a packet is lost. The dotted lines show the time that would be taken by the transmission of a packet and its acknowledgement, if the packet was not lost.

13.5 The Idea Behind Sliding Windows

Before examining the TCP stream service, we need to explore an additional concept that underlies stream transmission. The concept, known as a *sliding window*, makes stream transmission efficient. To understand the motivation for sliding windows, recall the sequence of events that Figure 13.1 depicts. To achieve reliability, the sender transmits a packet and then waits for an acknowledgement before transmitting another. As Figure 13.1 shows, data only flows between the machines in one direction at any time, even if the network is capable of simultaneous communication in both directions. The network will be completely idle during times that machines delay responses (e.g., while machines compute routes or checksums). If we imagine a network with high transmission delays, the problem becomes clear:

> *A simple positive acknowledgement protocol wastes a substantial amount of network bandwidth because it must delay sending a new packet until it receives an acknowledgement for the previous packet.*

The sliding window technique is a more complex form of positive acknowledgement and retransmission than the simple method discussed above. Sliding window protocols use network bandwidth better because they allow the sender to transmit multiple packets before waiting for an acknowledgement. The easiest way to envision sliding

window operation is to think of a sequence of packets to be transmitted as Figure 13.3 shows. The protocol places a small, fixed-size *window* on the sequence and transmits all packets that lie inside the window.

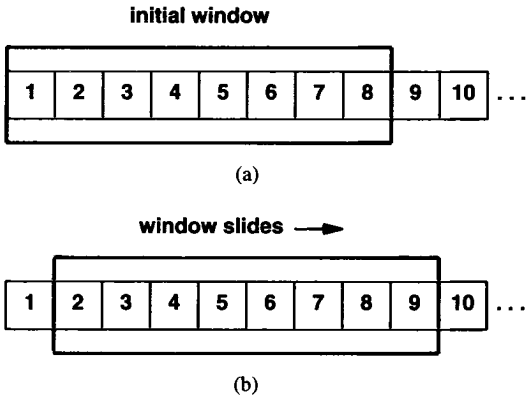

Figure 13.3 (a) A sliding window protocol with eight packets in the window, and (b) The window sliding so that packet *9* can be sent when an acknowledgement has been received for packet *1*. Only unacknowledged packets are retransmitted.

We say that a packet is *unacknowledged* if it has been transmitted but no acknowledgement has been received. Technically, the number of packets that can be unacknowledged at any given time is constrained by the *window size* and is limited to a small, fixed number. For example, in a sliding window protocol with window size *8*, the sender is permitted to transmit *8* packets before it receives an acknowledgement.

As Figure 13.3 shows, once the sender receives an acknowledgement for the first packet inside the window, it "slides" the window along and sends the next packet. The window continues to slide as long as acknowledgements are received.

The performance of sliding window protocols depends on the window size and the speed at which the network accepts packets. Figure 13.4 shows an example of the operation of a sliding window protocol when sending three packets. Note that the sender transmits all three packets before receiving any acknowledgements.

With a window size of *1*, a sliding window protocol is exactly the same as our simple positive acknowledgement protocol. By increasing the window size, it is possible to eliminate network idle time completely. That is, in the steady state, the sender can transmit packets as fast as the network can transfer them. The main point is:

> *Because a well tuned sliding window protocol keeps the network completely saturated with packets, it obtains substantially higher throughput than a simple positive acknowledgement protocol.*

Conceptually, a sliding window protocol always remembers which packets have been acknowledged and keeps a separate timer for each unacknowledged packet. If a packet is lost, the timer expires and the sender retransmits that packet. When the sender slides its window, it moves past all acknowledged packets. At the receiving end, the protocol software keeps an analogous window, accepting and acknowledging packets as they arrive. Thus, the window partitions the sequence of packets into three sets: those packets to the left of the window have been successfully transmitted, received, and acknowledged; those packets to the right have not yet been transmitted; and those packets that lie in the window are being transmitted. The lowest numbered packet in the window is the first packet in the sequence that has not been acknowledged.

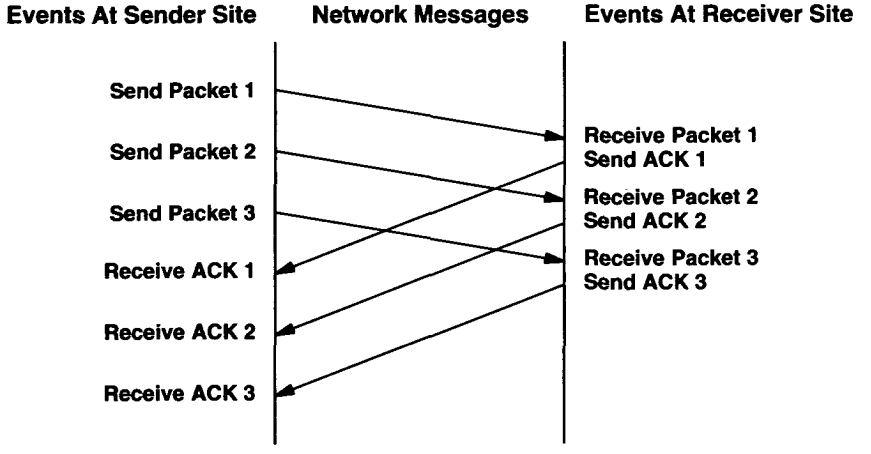

Figure 13.4 An example of three packets transmitted using a sliding window protocol. The key concept is that the sender can transmit all packets in the window without waiting for an acknowledgement.

13.6 The Transmission Control Protocol

Now that we understand the principle of sliding windows, we can examine the reliable stream service provided by the TCP/IP Internet protocol suite. The service is defined by the *Transmission Control Protocol*, or *TCP*. The reliable stream service is so important that the entire protocol suite is referred to as TCP/IP. It is important to understand that:

TCP is a communication protocol, not a piece of software.

The difference between a protocol and the software that implements it is analogous to the difference between the definition of a programming language and a compiler. As in the programming language world, the distinction between definition and implementa-

tion sometimes becomes blurred. People encounter TCP software much more frequently than they encounter the protocol specification, so it is natural to think of a particular implementation as the standard. Nevertheless, the reader should try to distinguish between the two.

Exactly what does TCP provide? TCP is complex, so there is no simple answer. The protocol specifies the format of the data and acknowledgements that two computers exchange to achieve a reliable transfer, as well as the procedures the computers use to ensure that the data arrives correctly. It specifies how TCP software distinguishes among multiple destinations on a given machine, and how communicating machines recover from errors like lost or duplicated packets. The protocol also specifies how two computers initiate a TCP stream transfer and how they agree when it is complete.

It is also important to understand what the protocol does not include. Although the TCP specification describes how application programs use TCP in general terms, it does not dictate the details of the interface between an application program and TCP. That is, the protocol documentation only discusses the operations TCP supplies; it does not specify the exact procedures application programs invoke to access these operations. The reason for leaving the application program interface unspecified is flexibility. In particular, because programmers usually implement TCP in the computer's operating system, they need to employ whatever interface the operating system supplies. Allowing the implementor flexibility makes it possible to have a single specification for TCP that can be used to build software for a variety of machines.

Because TCP assumes little about the underlying communication system, TCP can be used with a variety of packet delivery systems, including the IP datagram delivery service. For example, TCP can be implemented to use dialup telephone lines, a local area network, a high speed fiber optic network, or a lower speed long haul network. In fact, the large variety of delivery systems TCP can use is one of its strengths.

13.7 Ports, Connections, And Endpoints

Like the User Datagram Protocol (UDP) presented in Chapter 12, TCP resides above IP in the protocol layering scheme. Figure 13.5 shows the conceptual organization. TCP allows multiple application programs on a given machine to communicate concurrently, and it demultiplexes incoming TCP traffic among application programs. Like the User Datagram Protocol, TCP uses *protocol port* numbers to identify the ultimate destination within a machine. Each port is assigned a small integer used to identify it†.

†Although both TCP and UDP use integer port identifiers starting at *1* to identify ports, there is no confusion between them because an incoming IP datagram identifies the protocol being used as well as the port

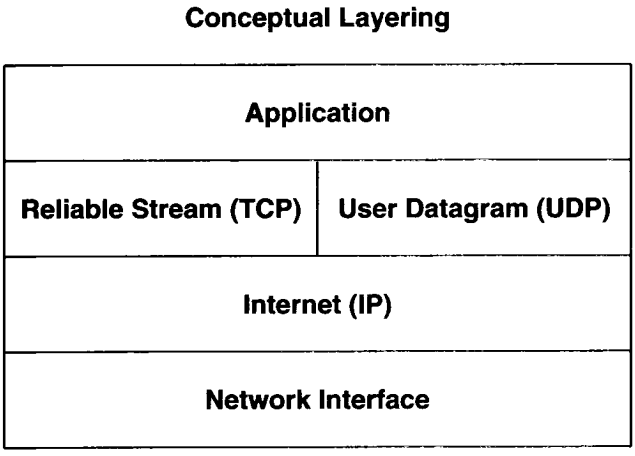

Figure 13.5 The conceptual layering of UDP and TCP above IP. TCP provides a reliable stream service, while UDP provides an unreliable datagram delivery service. Application programs use both.

When we discussed UDP ports, we said to think of each port as a queue into which protocol software places arriving datagrams. TCP ports are much more complex because a given port number does not correspond to a single object. Instead, TCP has been built on the *connection abstraction*, in which the objects to be identified are virtual circuit connections, not individual ports. Understanding that TCP uses the notion of connections is crucial because it helps explain the meaning and use of TCP port numbers:

> *TCP uses the connection, not the protocol port, as its fundamental abstraction; connections are identified by a pair of endpoints.*

Exactly what are the "endpoints" of a connection? We have said that a connection consists of a virtual circuit between two application programs, so it might be natural to assume that an application program serves as the connection "endpoint." It is not. Instead, TCP defines an *endpoint* to be a pair of integers (*host*, *port*), where *host* is the IP address for a host and *port* is a TCP port on that host. For example, the endpoint (*128.10.2.3*, *25*) specifies TCP port *25* on the machine with IP address *128.10.2.3*.

Now that we have defined endpoints, it will be easy to understand connections. Recall that a connection is defined by its two endpoints. Thus, if there is a connection from machine (*18.26.0.36*) at MIT to machine (*128.10.2.3*) at Purdue University, it might be defined by the endpoints:

(*18.26.0.36*, *1069*) and (*128.10.2.3*, *25*).

Meanwhile, another connection might be in progress from machine (*128.9.0.32*) at the Information Sciences Institute to the same machine at Purdue, identified by its endpoints:

(*128.9.0.32*, *1184*) and (*128.10.2.3*, *53*).

So far, our examples of connections have been straightforward because the ports used at all endpoints have been unique. However, the connection abstraction allows multiple connections to share an endpoint. For example, we could add another connection to the two listed above from machine (*128.2.254.139*) at CMU to the machine at Purdue:

(*128.2.254.139*, *1184*) and (*128.10.2.3*, *53*).

It might seem strange that two connections can use the TCP port *53* on machine 128.10.2.3 simultaneously, but there is no ambiguity. Because TCP associates incoming messages with a connection instead of a protocol port, it uses both endpoints to identify the appropriate connection. The important idea to remember is:

> *Because TCP identifies a connection by a pair of endpoints, a given TCP port number can be shared by multiple connections on the same machine.*

From a programmer's point of view, the connection abstraction is significant. It means a programmer can devise a program that provides concurrent service to multiple connections simultaneously without needing unique local port numbers for each connection. For example, most systems provide concurrent access to their electronic mail service, allowing multiple computers to send them electronic mail concurrently. Because the program that accepts incoming mail uses TCP to communicate, it only needs to use one local TCP port even though it allows multiple connections to proceed concurrently.

13.8 Passive And Active Opens

Unlike UDP, TCP is a connection-oriented protocol that requires both endpoints to agree to participate. That is, before TCP traffic can pass across an internet, application programs at both ends of the connection must agree that the connection is desired. To do so, the application program on one end performs a *passive open* function by contacting its operating system and indicating that it will accept an incoming connection. At that time, the operating system assigns a TCP port number for its end of the connection. The application program at the other end must then contact its operating system using an *active open* request to establish a connection. The two TCP software modules communicate to establish and verify a connection. Once a connection has been created, application programs can begin to pass data; the TCP software modules at each end exchange messages that guarantee reliable delivery. We will return to the details of establishing connections after examining the TCP message format.

13.9 Segments, Streams, And Sequence Numbers

TCP views the data stream as a sequence of octets or bytes that it divides into *segments* for transmission. Usually, each segment travels across an internet in a single IP datagram.

TCP uses a specialized sliding window mechanism to solve two important problems: efficient transmission and flow control. Like the sliding window protocol described earlier, the TCP window mechanism makes it possible to send multiple segments before an acknowledgement arrives. Doing so increases total throughput because it keeps the network busy. The TCP form of a sliding window protocol also solves the end-to-end *flow control* problem, by allowing the receiver to restrict transmission until it has sufficient buffer space to accommodate more data.

The TCP sliding window mechanism operates at the octet level, not at the segment or packet level. Octets of the data stream are numbered sequentially, and a sender keeps three pointers associated with every connection. The pointers define a sliding window as Figure 13.6 illustrates. The first pointer marks the left of the sliding window, separating octets that have been sent and acknowledged from octets yet to be acknowledged. A second pointer marks the right of the sliding window and defines the highest octet in the sequence that can be sent before more acknowledgements are received. The third pointer marks the boundary inside the window that separates those octets that have already been sent from those octets that have not been sent. The protocol software sends all octets in the window without delay, so the boundary inside the window usually moves from left to right quickly.

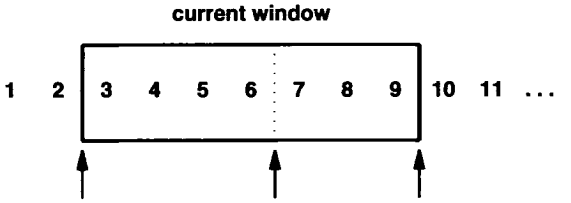

Figure 13.6 An example of the TCP sliding window. Octets through 2 have been sent and acknowledged, octets 3 through 6 have been sent but not acknowledged, octets 7 though 9 have not been sent but will be sent without delay, and octets 10 and higher cannot be sent until the window moves.

We have described how the sender's TCP window slides along and mentioned that the receiver must maintain a similar window to piece the stream together again. It is important to understand, however, that because TCP connections are full duplex, two transfers proceed simultaneously over each connection, one in each direction. We think of the transfers as completely independent because at any time data can flow across the connection in one direction, or in both directions. Thus, TCP software at each end

maintains two windows per connection (for a total of four), one slides along the data stream being sent, while the other slides along as data is received.

13.10 Variable Window Size And Flow Control

One difference between the TCP sliding window protocol and the simplified sliding window protocol presented earlier occurs because TCP allows the window size to vary over time. Each acknowledgement, which specifies how many octets have been received, contains a *window advertisement* that specifies how many additional octets of data the receiver is prepared to accept. We think of the window advertisement as specifying the receiver's current buffer size. In response to an increased window advertisement, the sender increases the size of its sliding window and proceeds to send octets that have not been acknowledged. In response to a decreased window advertisement, the sender decreases the size of its window and stops sending octets beyond the boundary. TCP software should not contradict previous advertisements by shrinking the window past previously acceptable positions in the octet stream. Instead, smaller advertisements accompany acknowledgements, so the window size changes at the time it slides forward.

The advantage of using a variable size window is that it provides flow control as well as reliable transfer. To avoid receiving more data than it can store, the receiver sends smaller window advertisements as its buffer fills. In the extreme case, the receiver advertises a window size of zero to stop all transmissions. Later, when buffer space becomes available, the receiver advertises a nonzero window size to trigger the flow of data again†.

Having a mechanism for flow control is essential in an internet environment, where machines of various speeds and sizes communicate through networks and routers of various speeds and capacities. There are two independent flow problems. First, internet protocols need end-to-end flow control between the source and ultimate destination. For example, when a minicomputer communicates with a large mainframe, the minicomputer needs to regulate the influx of data, or protocol software would be overrun quickly. Thus, TCP must implement end-to-end flow control to guarantee reliable delivery. Second, internet protocols need a flow control mechanism that allows intermediate systems (i.e., routers) to control a source that sends more traffic than the machine can tolerate.

When intermediate machines become overloaded, the condition is called *congestion*, and mechanisms to solve the problem are called *congestion control* mechanisms. TCP uses its sliding window scheme to solve the end-to-end flow control problem; it does not have an explicit mechanism for congestion control. We will see later, however, that a carefully programmed TCP implementation can detect and recover from congestion while a poor implementation can make it worse. In particular, although a carefully chosen retransmission scheme can help avoid congestion, a poorly chosen scheme can exacerbate it.

†There are two exceptions to transmission when the window size is zero. First, a sender is allowed to transmit a segment with the urgent bit set to inform the receiver that urgent data is available. Second, to avoid a potential deadlock that can arise if a nonzero advertisement is lost after the window size reaches zero, the sender probes a zero sized window periodically.

13.11 TCP Segment Format

The unit of transfer between the TCP software on two machines is called a *segment*. Segments are exchanged to establish connections, transfer data, send acknowledgements, advertise window sizes, and close connections. Because TCP uses piggybacking, an acknowledgement traveling from machine *A* to machine *B* may travel in the same segment as data traveling from machine *A* to machine *B*, even though the acknowledgement refers to data sent from *B* to *A*†. Figure 13.7 shows the TCP segment format.

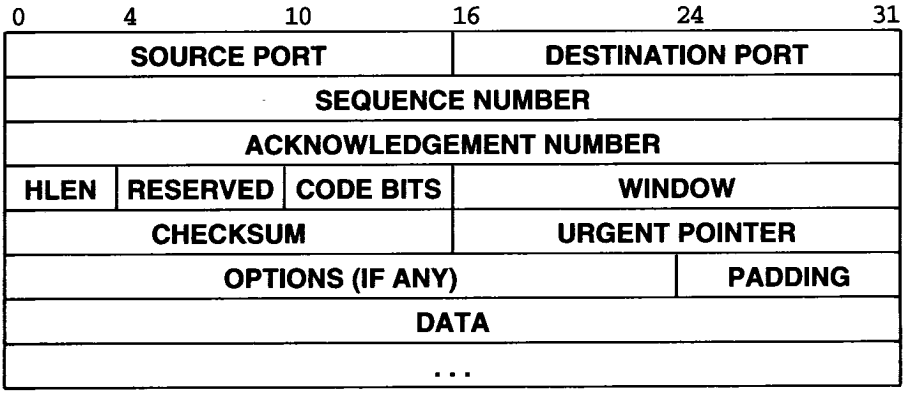

Figure 13.7 The format of a TCP segment with a TCP header followed by data. Segments are used to establish connections as well as to carry data and acknowledgements.

Each segment is divided into two parts, a header followed by data. The header, known as the *TCP header*, carries the expected identification and control information. Fields *SOURCE PORT* and *DESTINATION PORT* contain the TCP port numbers that identify the application programs at the ends of the connection. The *SEQUENCE NUMBER* field identifies the position in the sender's byte stream of the data in the segment. The *ACKNOWLEDGEMENT NUMBER* field identifies the number of the octet that the source expects to receive next. Note that the sequence number refers to the stream flowing in the same direction as the segment, while the acknowledgement number refers to the stream flowing in the opposite direction from the segment.

The *HLEN*‡ field contains an integer that specifies the length of the segment header measured in 32-bit multiples. It is needed because the *OPTIONS* field varies in length, depending on which options have been included. Thus, the size of the TCP header varies depending on the options selected. The 6-bit field marked *RESERVED* is reserved for future use.

†In practice, piggybacking does not usually occur because most applications do not send data in both directions simultaneously.

‡The specification says the *HLEN* field is the *offset* of the data area within the segment.

Some segments carry only an acknowledgement while some carry data. Others carry requests to establish or close a connection. TCP software uses the 6-bit field labeled *CODE BITS* to determine the purpose and contents of the segment. The six bits tell how to interpret other fields in the header according to the table in Figure 13.8.

Bit (left to right)	Meaning if bit set to 1
URG	Urgent pointer field is valid
ACK	Acknowledgement field is valid
PSH	This segment requests a push
RST	Reset the connection
SYN	Synchronize sequence numbers
FIN	Sender has reached end of its byte stream

Figure 13.8 Bits of the CODE field in the TCP header.

TCP software advertises how much data it is willing to accept every time it sends a segment by specifying its buffer size in the *WINDOW* field. The field contains a 16-bit unsigned integer in network-standard byte order. Window advertisements provide another example of piggybacking because they accompany all segments, including those carrying data as well as those carrying only an acknowledgement.

13.12 Out Of Band Data

Although TCP is a stream-oriented protocol, it is sometimes important for the program at one end of a connection to send data *out of band*, without waiting for the program at the other end of the connection to consume octets already in the stream. For example, when TCP is used for a remote login session, the user may decide to send a keyboard sequence that *interrupts* or *aborts* the program at the other end. Such signals are most often needed when a program on the remote machine fails to operate correctly. The signals must be sent without waiting for the program to read octets already in the TCP stream (or one would not be able to abort programs that stop reading input).

To accommodate out of band signaling, TCP allows the sender to specify data as *urgent*, meaning that the receiving program should be notified of its arrival as quickly as possible, regardless of its position in the stream. The protocol specifies that when urgent data is found, the receiving TCP should notify whatever application program is associated with the connection to go into "urgent mode." After all urgent data has been consumed, TCP tells the application program to return to normal operation.

The exact details of how TCP informs the application program about urgent data depend on the computer's operating system, of course. The mechanism used to mark urgent data when transmitting it in a segment consists of the URG code bit and the *URGENT POINTER* field. When the URG bit is set, the urgent pointer specifies the position in the segment where urgent data ends.

13.13 Maximum Segment Size Option

Not all segments sent across a connection will be of the same size. However, both ends need to agree on a maximum segment they will transfer. TCP software uses the *OPTIONS* field to negotiate with the TCP software at the other end of the connection; one of the options allows TCP software to specify the *maximum segment size* (*MSS*) that it is willing to receive. For example, when an embedded system that only has a few hundred bytes of buffer space connects to a large supercomputer, it can negotiate an MSS that restricts segments so they fit in the buffer. It is especially important for computers connected by high-speed local area networks to choose a maximum segment size that fills packets or they will not make good use of the bandwidth. Therefore, if the two endpoints lie on the same physical network, TCP usually computes a maximum segment size such that the resulting IP datagrams will match the network MTU. If the endpoints do not lie on the same physical network, they can attempt to discover the minimum MTU along the path between them, or choose a maximum segment size of *536* (the default size of an IP datagram, *576*, minus the standard size of IP and TCP headers).

In a general internet environment, choosing a good maximum segment size can be difficult because performance can be poor for either extremely large segment sizes or extremely small sizes. On one hand, when the segment size is small, network utilization remains low. To see why, recall that TCP segments travel encapsulated in IP datagrams which are encapsulated in physical network frames. Thus, each segment has at least 40 octets of TCP and IP headers in addition to the data. Therefore, datagrams carrying only one octet of data use at most 1/41 of the underlying network bandwidth for user data; in practice, minimum interpacket gaps and network hardware framing bits make the ratio even smaller.

On the other hand, extremely large segment sizes can also produce poor performance. Large segments result in large IP datagrams. When such datagrams travel across a network with small MTU, IP must fragment them. Unlike a TCP segment, a fragment cannot be acknowledged or retransmitted independently; all fragments must arrive or the entire datagram must be retransmitted. Because the probability of losing a given fragment is nonzero, increasing segment size above the fragmentation threshold decreases the probability the datagram will arrive, which decreases throughput.

In theory, the optimum segment size, *S*, occurs when the IP datagrams carrying the segments are as large as possible without requiring fragmentation anywhere along the path from the source to the destination. In practice, finding *S* is difficult for several reasons. First, most implementations of TCP do not include a mechanism for doing so†. Second, because routers in an internet can change routes dynamically, the path datagrams follow between a pair of communicating computers can change dynamically and so can the size at which datagrams must be fragmented. Third, the optimum size depends on lower-level protocol headers (e.g., the segment size must be reduced to accommodate IP options). Research on the problem of finding an optimal segment size continues.

†To discover the path MTU, a sender probes the path by sending datagrams with the IP *do not fragment* bit set. It then decreases the size if ICMP error messages report that fragmentation was required.

13.14 TCP Checksum Computation

The *CHECKSUM* field in the TCP header contains a 16-bit integer checksum used to verify the integrity of the data as well as the TCP header. To compute the checksum, TCP software on the sending machine follows a procedure like the one described in Chapter 12 for UDP. It prepends a *pseudo header* to the segment, appends enough zero bits to make the segment a multiple of 16 bits, and computes the 16-bit checksum over the entire result. TCP does not count the pseudo header or padding in the segment length, nor does it transmit them. Also, it assumes the checksum field itself is zero for purposes of the checksum computation. As with other checksums, TCP uses 16-bit arithmetic and takes the one's complement of the one's complement sum. At the receiving site, TCP software performs the same computation to verify that the segment arrived intact.

The purpose of using a pseudo header is exactly the same as in UDP. It allows the receiver to verify that the segment has reached its correct destination, which includes both a host IP address as well as a protocol port number. Both the source and destination IP addresses are important to TCP because it must use them to identify a connection to which the segment belongs. Therefore, whenever a datagram arrives carrying a TCP segment, IP must pass to TCP the source and destination IP addresses from the datagram as well as the segment itself. Figure 13.9 shows the format of the pseudo header used in the checksum computation.

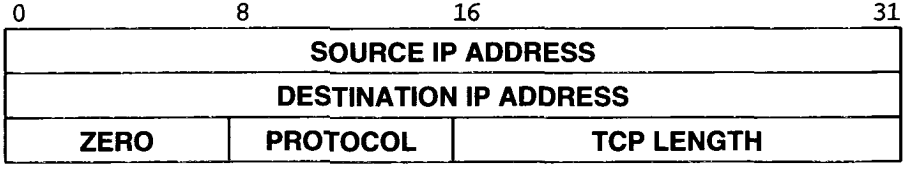

Figure 13.9 The format of the pseudo header used in TCP checksum computations. At the receiving site, this information is extracted from the IP datagram that carried the segment.

The sending TCP assigns field *PROTOCOL* the value that the underlying delivery system will use in its protocol type field. For IP datagrams carrying TCP, the value is *6*. The *TCP LENGTH* field specifies the total length of the TCP segment including the TCP header. At the receiving end, information used in the pseudo header is extracted from the IP datagram that carried the segment and included in the checksum computation to verify that the segment arrived at the correct destination intact.

13.15 Acknowledgements And Retransmission

Because TCP sends data in variable length segments and because retransmitted segments can include more data than the original, acknowledgements cannot easily refer to datagrams or segments. Instead, they refer to a position in the stream using the stream sequence numbers. The receiver collects data octets from arriving segments and reconstructs an exact copy of the stream being sent. Because segments travel in IP datagrams, they can be lost or delivered out of order; the receiver uses the sequence numbers to reorder segments. At any time, the receiver will have reconstructed zero or more octets contiguously from the beginning of the stream, but may have additional pieces of the stream from datagrams that arrived out of order. The receiver always acknowledges the longest contiguous prefix of the stream that has been received correctly. Each acknowledgement specifies a sequence value one greater than the highest octet position in the contiguous prefix it received. Thus, the sender receives continuous feedback from the receiver as it progresses through the stream. We can summarize this important idea:

> *A TCP acknowledgement specifies the sequence number of the next octet that the receiver expects to receive.*

The TCP acknowledgement scheme is called *cumulative* because it reports how much of the stream has accumulated. Cumulative acknowledgements have both advantages and disadvantages. One advantage is that acknowledgements are both easy to generate and unambiguous. Another advantage is that lost acknowledgements do not necessarily force retransmission. A major disadvantage is that the sender does not receive information about all successful transmissions, but only about a single position in the stream that has been received.

To understand why lack of information about all successful transmissions makes cumulative acknowledgements less efficient, think of a window that spans *5000* octets starting at position *101* in the stream, and suppose the sender has transmitted all data in the window by sending five segments. Suppose further that the first segment is lost, but all others arrive intact. As each segment arrives, the receiver sends an acknowledgement, but each acknowledgement specifies octet *101*, the next highest contiguous octet it expects to receive. There is no way for the receiver to tell the sender that most of the data for the current window has arrived.

When a timeout occurs at the sender's side, the sender must choose between two potentially inefficient schemes. It may choose to retransmit one segment or all five segments. In this case retransmitting all five segments is inefficient. When the first segment arrives, the receiver will have all the data in the window, and will acknowledge *5101*. If the sender follows the accepted standard and retransmits only the first unacknowledged segment, it must wait for the acknowledgement before it can decide what and how much to send. Thus, it reverts to a simple positive acknowledgement protocol and may lose the advantages of having a large window.

13.16 Timeout And Retransmission

One of the most important and complex ideas in TCP is embedded in the way it handles timeout and retransmission. Like other reliable protocols, TCP expects the destination to send acknowledgements whenever it successfully receives new octets from the data stream. Every time it sends a segment, TCP starts a timer and waits for an acknowledgement. If the timer expires before data in the segment has been acknowledged, TCP assumes that the segment was lost or corrupted and retransmits it.

To understand why the TCP retransmission algorithm differs from the algorithm used in many network protocols, we need to remember that TCP is intended for use in an internet environment. In an internet, a segment traveling between a pair of machines may traverse a single, low-delay network (e.g., a high-speed LAN), or it may travel across multiple intermediate networks through multiple routers. Thus, it is impossible to know *a priori* how quickly acknowledgements will return to the source. Furthermore, the delay at each router depends on traffic, so the total time required for a segment to travel to the destination and an acknowledgement to return to the source varies dramatically from one instant to another. Figure 13.10, which shows measurements of round trip times across the global Internet for 100 consecutive packets, illustrates the problem. TCP software must accommodate both the vast differences in the time required to reach various destinations and the changes in time required to reach a given destination as traffic load varies.

TCP accommodates varying internet delays by using an *adaptive retransmission algorithm*. In essence, TCP monitors the performance of each connection and deduces reasonable values for timeouts. As the performance of a connection changes, TCP revises its timeout value (i.e., it adapts to the change).

To collect the data needed for an adaptive algorithm, TCP records the time at which each segment is sent and the time at which an acknowledgement arrives for the data in that segment. From the two times, TCP computes an elapsed time known as a *sample round trip time* or *round trip sample*. Whenever it obtains a new round trip sample, TCP adjusts its notion of the average round trip time for the connection. Usually, TCP software stores the estimated round trip time, *RTT*, as a weighted average and uses new round trip samples to change the average slowly. For example, when computing a new weighted average, one early averaging technique used a constant weighting factor, α, where $0 \leq \alpha < 1$, to weight the old average against the latest round trip sample:

$$RTT = (\alpha * Old\_RTT) + ((1-\alpha) * New\_Round\_Trip\_Sample)$$

Choosing a value for α close to *1* makes the weighted average immune to changes that last a short time (e.g., a single segment that encounters long delay). Choosing a value for α close to *0* makes the weighted average respond to changes in delay very quickly.

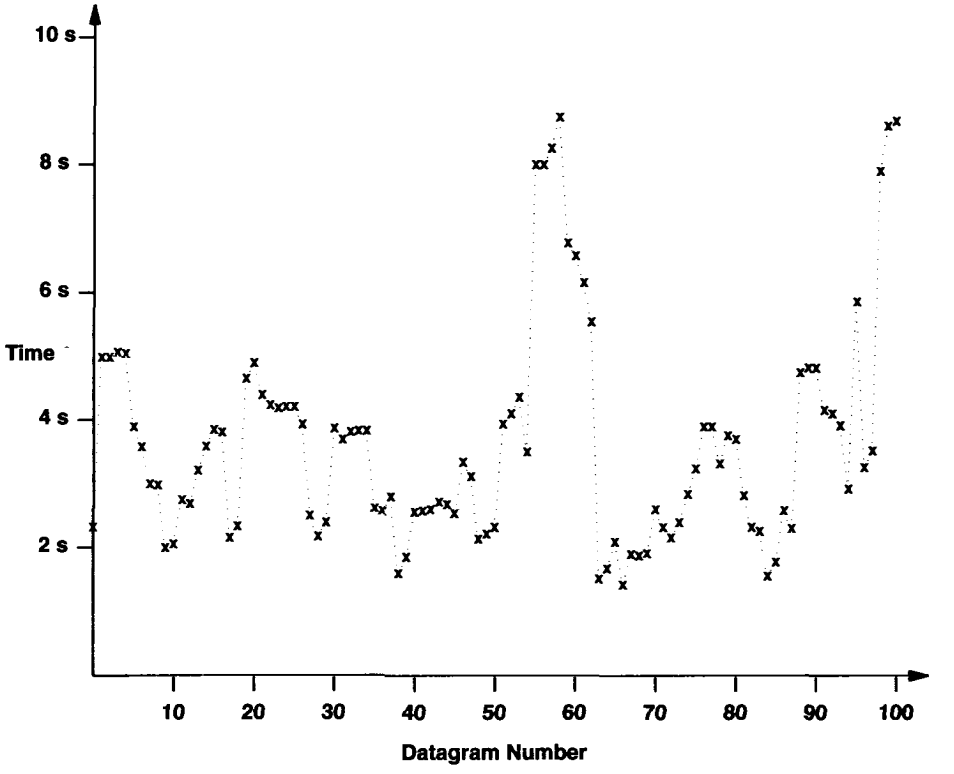

Figure 13.10 A plot of Internet round trip times as measured for 100 successive IP datagrams. Although the Internet now operates with much lower delay, the delays still vary over time.

When it sends a packet, TCP computes a timeout value as a function of the current round trip estimate. Early implementations of TCP used a constant weighting factor, β ($\beta > 1$), and made the timeout greater than the current round trip estimate:

$$\text{Timeout} = \beta * \text{RTT}$$

Choosing a value for β can be difficult. On one hand, to detect packet loss quickly, the timeout value should be close to the current round trip time (i.e., β should be close to *1*). Detecting packet loss quickly improves throughput because TCP will not wait an unnecessarily long time before retransmitting. On the other hand, if $\beta = 1$, TCP is overly eager — any small delay will cause an unnecessary retransmission, which wastes network bandwidth. The original specification recommended setting $\beta = 2$; more recent work described below has produced better techniques for adjusting timeout.

We can summarize the ideas presented so far:

> *To accommodate the varying delays encountered in an internet environment, TCP uses an adaptive retransmission algorithm that monitors delays on each connection and adjusts its timeout parameter accordingly.*

13.17 Accurate Measurement Of Round Trip Samples

In theory, measuring a round trip sample is trivial — it consists of subtracting the time at which the segment is sent from the time at which the acknowledgement arrives. However, complications arise because TCP uses a cumulative acknowledgement scheme in which an acknowledgement refers to data received, and not to the instance of a specific datagram that carried the data. Consider a retransmission. TCP forms a segment, places it in a datagram and sends it, the timer expires, and TCP sends the segment again in a second datagram. Because both datagrams carry exactly the same data, the sender has no way of knowing whether an acknowledgement corresponds to the original or retransmitted datagram. This phenomenon has been called *acknowledgement ambiguity*, and TCP acknowledgements are said to be *ambiguous*.

Should TCP assume acknowledgements belong with the earliest (i.e., original) transmission or the latest (i.e., the most recent retransmission)? Surprisingly, neither assumption works. Associating the acknowledgement with the original transmission can make the estimated round trip time grow without bound in cases where an internet loses datagrams†. If an acknowledgement arrives after one or more retransmissions, TCP will measure the round trip sample from the original transmission, and compute a new RTT using the excessively long sample. Thus, RTT will grow slightly. The next time TCP sends a segment, the larger RTT will result in slightly longer timeouts, so if an acknowledgement arrives after one or more retransmissions, the next sample round trip time will be even larger, and so on.

Associating the acknowledgement with the most recent retransmission can also fail. Consider what happens when the end-to-end delay suddenly increases. When TCP sends a segment, it uses the old round trip estimate to compute a timeout, which is now too small. The segment arrives and an acknowledgement starts back, but the increase in delay means the timer expires before the acknowledgement arrives, and TCP retransmits the segment. Shortly after TCP retransmits, the first acknowledgement arrives and is associated with the retransmission. The round trip sample will be much too small and will result in a slight decrease of the estimated round trip time, RTT. Unfortunately, lowering the estimated round trip time guarantees that TCP will set the timeout too small for the next segment. Ultimately, the estimated round trip time can stabilize at a value, T, such that the correct round trip time is slightly longer than some multiple of T. Implementations of TCP that associate acknowledgements with the most recent retransmission have been observed in a stable state with RTT slightly less than one-half of the correct value (i.e., TCP sends each segment exactly twice even though no loss occurs).

†The estimate can only grow arbitrarily large if every segment is lost at least once.

13.18 Karn's Algorithm And Timer Backoff

If the original transmission and the most recent transmission both fail to provide accurate round trip times, what should TCP do? The accepted answer is simple: TCP should not update the round trip estimate for retransmitted segments. This idea, known as *Karn's Algorithm*, avoids the problem of ambiguous acknowledgements altogether by only adjusting the estimated round trip for unambiguous acknowledgements (acknowledgements that arrive for segments that have only been transmitted once).

Of course, a simplistic implementation of Karn's algorithm, one that merely ignores times from retransmitted segments, can lead to failure as well. Consider what happens when TCP sends a segment after a sharp increase in delay. TCP computes a timeout using the existing round trip estimate. The timeout will be too small for the new delay and will force retransmission. If TCP ignores acknowledgements from retransmitted segments, it will never update the estimate and the cycle will continue.

To accommodate such failures, Karn's algorithm requires the sender to combine retransmission timeouts with a *timer backoff* strategy. The backoff technique computes an initial timeout using a formula like the one shown above. However, if the timer expires and causes a retransmission, TCP increases the timeout. In fact, each time it must retransmit a segment, TCP increases the timeout (to keep timeouts from becoming ridiculously long, most implementations limit increases to an upper bound that is larger than the delay along any path in the internet).

Implementations use a variety of techniques to compute backoff. Most choose a multiplicative factor, γ, and set the new value to:

$$\text{new\_timeout} = \gamma * \text{timeout}$$

Typically, γ is 2. (It has been argued that values of γ less than 2 lead to instabilities.) Other implementations use a table of multiplicative factors, allowing arbitrary backoff at each step†.

Karn's algorithm combines the backoff technique with round trip estimation to solve the problem of never increasing round trip estimates:

> *Karn's algorithm: When computing the round trip estimate, ignore samples that correspond to retransmitted segments, but use a backoff strategy, and retain the timeout value from a retransmitted packet for subsequent packets until a valid sample is obtained.*

Generally speaking, when an internet misbehaves, Karn's algorithm separates computation of the timeout value from the current round trip estimate. It uses the round trip estimate to compute an initial timeout value, but then backs off the timeout on each retransmission until it can successfully transfer a segment. When it sends subsequent segments, it retains the timeout value that results from backoff. Finally, when an acknowledgement arrives corresponding to a segment that did not require retransmission,

†Berkeley UNIX is the most notable system that uses a table of factors, but current values in the table are equivalent to using γ=2.

TCP recomputes the round trip estimate and resets the timeout accordingly. Experience shows that Karn's algorithm works well even in networks with high packet loss†.

13.19 Responding To High Variance In Delay

Research into round trip estimation has shown that the computations described above do not adapt to a wide range of variation in delay. Queueing theory suggests that the variation in round trip time, σ, varies proportional to $1/(1-L)$, where L is the current network load, $0 \leq L \leq 1$. If an internet is running at 50% of capacity, we expect the round trip delay to vary by a factor of $\pm 2\sigma$, or *4*. When the load reaches 80%, we expect a variation of *10*. The original TCP standard specified the technique for estimating round trip time that we described earlier. Using that technique and limiting β to the suggested value of *2* means the round trip estimation can adapt to loads of at most 30%.

The 1989 specification for TCP requires implementations to estimate both the average round trip time and the variance, and to use the estimated variance in place of the constant β. As a result, new implementations of TCP can adapt to a wider range of variation in delay and yield substantially higher throughput. Fortunately, the approximations require little computation; extremely efficient programs can be derived from the following simple equations:

$$\text{DIFF} = \text{SAMPLE} - \text{Old\_RTT}$$

$$\text{Smoothed\_RTT} = \text{Old\_RTT} + \delta * \text{DIFF}$$

$$\text{DEV} = \text{Old\_DEV} + \rho\,(|\text{DIFF}| - \text{Old\_DEV})$$

$$\text{Timeout} = \text{Smoothed\_RTT} + \eta * \text{DEV}$$

where *DEV* is the estimated mean deviation, δ is a fraction between *0* and *1* that controls how quickly the new sample affects the weighted average, ρ is a fraction between *0* and *1* that controls how quickly the new sample affects the mean deviation, and η is a factor that controls how much the deviation affects the round trip timeout. To make the computation efficient, TCP chooses δ and ρ to each be an inverse of a power of *2*, scales the computation by 2^n for an appropriate n, and uses integer arithmetic. Research suggests values of $\delta = 1/2^3$, $\rho = 1/2^2$, and $n = 3$ will work well. The original value for η in 4.3BSD UNIX was *2*; it was changed to *4* in 4.4 BSD UNIX.

Figure 13.11 uses a set of randomly generated values to illustrate how the computed timeout changes as the roundtrip time varies. Although the roundtrip times are artificial, they follow a pattern observed in practice: successive packets show small variations in delay as the overall average rises or falls.

†Phil Karn is an amateur radio enthusiast who developed this algorithm to allow TCP communication across a high-loss packet radio connection.

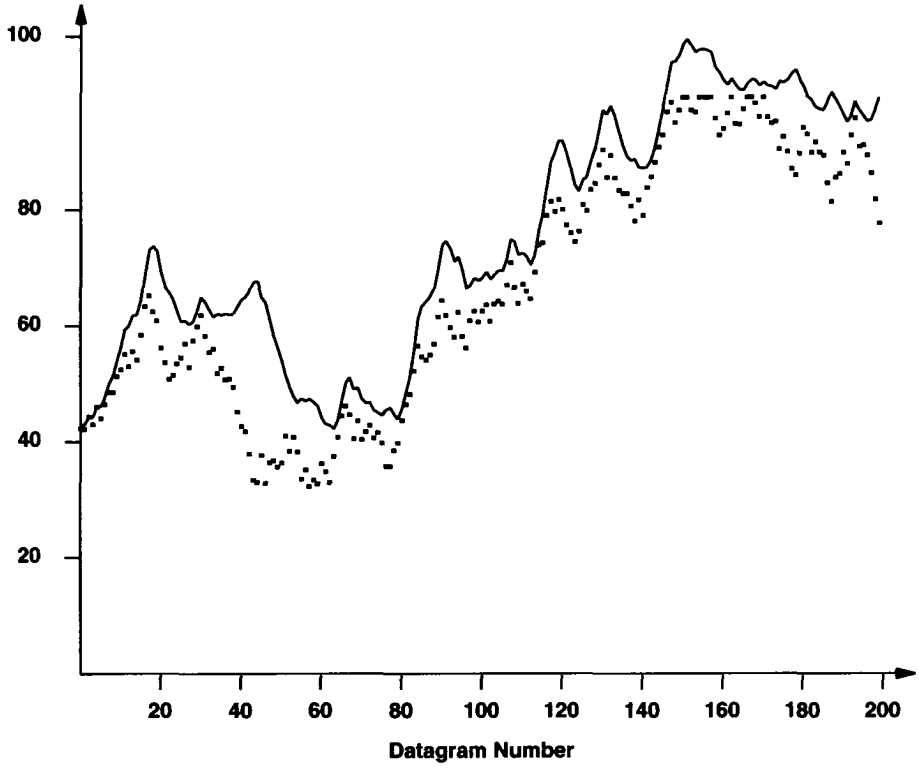

Figure 13.11 A set of 200 (randomly generated) roundtrip times shown as dots, and the TCP retransmission timer shown as a solid line. The timeout increases when delay varies.

Note that frequent change in the roundtrip time, including a cycle of increase and decrease, can produce an increase in the retransmission timer. Furthermore, although the timer tends to increase quickly when delay rises, it does not decrease as rapidly when delay falls.

Figure 13.12 uses the data points from Figure 13.10 to show how TCP responds to the extreme case of variance in delay. Recall that the goal is to have the retransmission timer estimate the actual roundtrip time as closely as possible without underestimating. The figure shows that although the timer responds quickly, it can underestimate. For example, between the two successive datagrams marked with arrows, the delay doubles from less than 4 seconds to more than 8. More important, the abrupt change follows a period of relative stability in which the variation in delay is small, making it impossible for any algorithm to anticipate the change. In the case of the TCP algorithm, because the timeout (approximately 5) substantially underestimates the large delay, an unnecessary retransmission occurs. However, the estimate responds quickly to the increase in delay, meaning that successive packets arrive without retransmission.

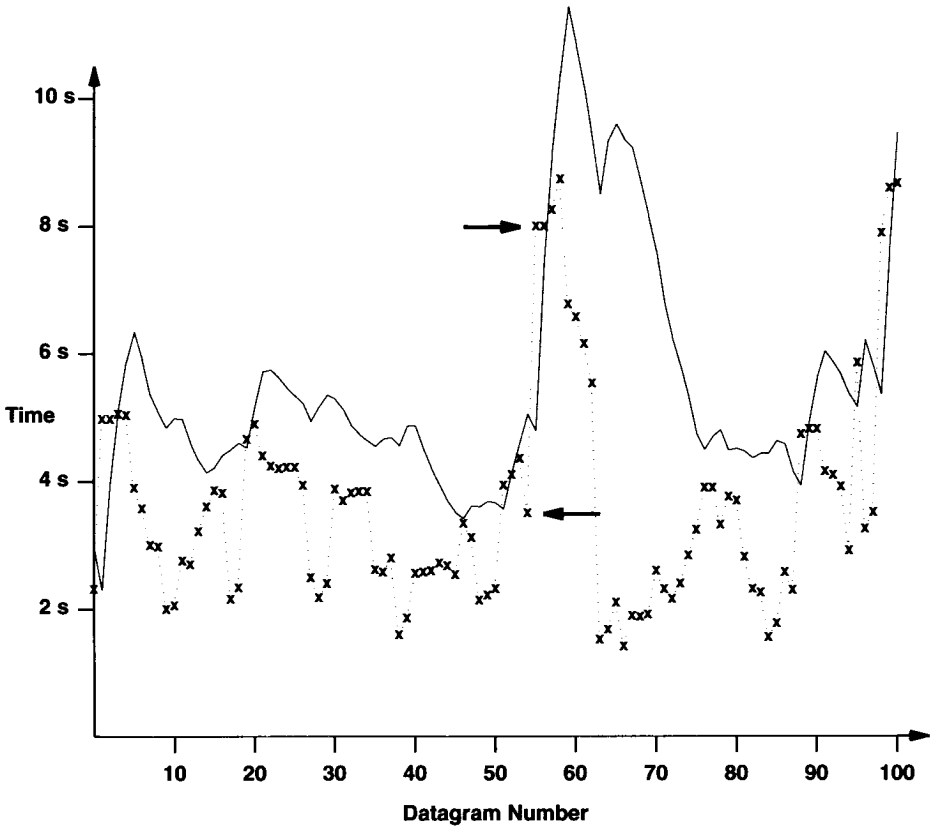

Figure 13.12 The TCP retransmission timer for the data from Figure 13.10. Arrows mark two successive datagrams where the delay doubles.

13.20 Response To Congestion

It may seem that TCP software could be designed by considering the interaction between the two endpoints of a connection and the communication delays between those endpoints. In practice, however, TCP must also react to *congestion* in the internet. Congestion is a condition of severe delay caused by an overload of datagrams at one or more switching points (e.g., at routers). When congestion occurs, delays increase and the router begins to enqueue datagrams until it can route them. We must remember that each router has finite storage capacity and that datagrams compete for that storage (i.e., in a datagram based internet, there is no preallocation of resources to individual TCP connections). In the worst case, the total number of datagrams arriving at the congested router grows until the router reaches capacity and starts to drop datagrams.

Endpoints do not usually know the details of where congestion has occurred or why. To them, congestion simply means increased delay. Unfortunately, most transport protocols use timeout and retransmission, so they respond to increased delay by retransmitting datagrams. Retransmissions aggravate congestion instead of alleviating it. If unchecked, the increased traffic will produce increased delay, leading to increased traffic, and so on, until the network becomes useless. The condition is known as *congestion collapse.*

To avoid congestion collapse, TCP must reduce transmission rates when congestion occurs. Routers watch queue lengths and use techniques like ICMP source quench to inform hosts that congestion has occurred†, but transport protocols like TCP can help avoid congestion by reducing transmission rates automatically whenever delays occur. Of course, algorithms to avoid congestion must be constructed carefully because even under normal operating conditions an internet will exhibit wide variation in round trip delays.

To avoid congestion, the TCP standard now recommends using two techniques: *slow-start* and *multiplicative decrease.* They are related and can be implemented easily. We said that for each connection, TCP must remember the size of the receiver's window (i.e., the buffer size advertised in acknowledgements). To control congestion TCP maintains a second limit, called the *congestion window limit* or *congestion window*, that it uses to restrict data flow to less than the receiver's buffer size when congestion occurs. That is, at any time, TCP acts as if the window size is:

Allowed_window = min(receiver_advertisement, congestion_window)

In the steady state on a non-congested connection, the congestion window is the same size as the receiver's window. Reducing the congestion window reduces the traffic TCP will inject into the connection. To estimate congestion window size, TCP assumes that most datagram loss comes from congestion and uses the following strategy:

> *Multiplicative Decrease Congestion Avoidance: Upon loss of a segment, reduce the congestion window by half (down to a minimum of at least one segment). For those segments that remain in the allowed window, backoff the retransmission timer exponentially.*

Because TCP reduces the congestion window by half for *every* loss, it decreases the window exponentially if loss continues. In other words, if congestion is likely, TCP reduces the volume of traffic exponentially and the rate of retransmission exponentially. If loss continues, TCP eventually limits transmission to a single datagram and continues to double timeout values before retransmitting. The idea is to provide quick and significant traffic reduction to allow routers enough time to clear the datagrams already in their queues.

How can TCP recover when congestion ends? You might suspect that TCP should reverse the multiplicative decrease and double the congestion window when traffic begins to flow again. However, doing so produces an unstable system that oscillates wild-

†In a congested network, queue lengths grow exponentially for a significant time.

ly between no traffic and congestion. Instead, TCP uses a technique called *slow-start*† to scale up transmission:

> *Slow-Start (Additive) Recovery: Whenever starting traffic on a new connection or increasing traffic after a period of congestion, start the congestion window at the size of a single segment and increase the congestion window by one segment each time an acknowledgement arrives.*

Slow-start avoids swamping the internet with additional traffic immediately after congestion clears or when new connections suddenly start.

The term *slow-start* may be a misnomer because under ideal conditions, the start is not very slow. TCP initializes the congestion window to *1*, sends an initial segment, and waits. When the acknowledgement arrives, it increases the congestion window to *2*, sends two segments, and waits. When the two acknowledgements arrive they each increase the congestion window by *1*, so TCP can send *4* segments. Acknowledgements for those will increase the congestion window to *8*. Within four round-trip times, TCP can send *16* segments, often enough to reach the receiver's window limit. Even for extremely large windows, it takes only $\log_2 N$ round trips before TCP can send N segments.

To avoid increasing the window size too quickly and causing additional congestion, TCP adds one additional restriction. Once the congestion window reaches one half of its original size before congestion, TCP enters a *congestion avoidance* phase and slows down the rate of increment. During congestion avoidance, it increases the congestion window by *1* only if all segments in the window have been acknowledged.

Taken together, slow-start increase, multiplicative decrease, congestion avoidance, measurement of variation, and exponential timer backoff improve the performance of TCP dramatically without adding any significant computational overhead to the protocol software. Versions of TCP that use these techniques have improved the performance of previous versions by factors of *2* to *10*.

13.21 Congestion, Tail Drop, And TCP

We said that communication protocols are divided into layers to make it possible for designers to focus on a single problem at a time. The separation of functionality into layers is both necessary and useful — it means that one layer can be changed without affecting other layers, but it means that layers operate in isolation. For example, because it operates end-to-end, TCP remains unchanged when the path between the endpoints changes (e.g., routes change or additional networks routers are added). However, the isolation of layers restricts inter-layer communication. In particular, although TCP on the original source interacts with TCP on the ultimate destination, it cannot interact with lower layer elements along the path. Thus, neither the sending nor receiving

†The term *slow-start* is attributed to John Nagle; the technique was originally called *soft-start*.

TCP receives reports about conditions in the network, nor does either end inform lower layers along the path before transferring data.

Researchers have observed that the lack of communication between layers means that the choice of policy or implementation at one layer can have a dramatic effect on the performance of higher layers. In the case of TCP, policies that routers use to handle datagrams can have a significant effect on both the performance of a single TCP connection and the aggregate throughput of all connections. For example, if a router delays some datagrams more than others†, TCP will back off its retransmission timer. If the delay exceeds the retransmission timeout, TCP will assume congestion has occurred. Thus, although each layer is defined independently, researchers try to devise mechanisms and implementations that work well with protocols in other layers.

The most important interaction between IP implementation policies and TCP occurs when a router becomes overrun and drops datagrams. Because a router places each incoming datagram in a queue in memory until it can be processed, the policy focuses on queue management. When datagrams arrive faster than they can be forwarded, the queue grows; when datagrams arrive slower than they can be forwarded, the queue shrinks. However, because memory is finite, the queue cannot grow without bound. Early router software used a *tail-drop* policy to manage queue overflow:

> *Tail-Drop Policy For Routers: if the input queue is filled when a datagram arrives, discard the datagram.*

The name *tail-drop* arises from the effect of the policy on an arriving sequence of datagrams. Once the queue fills, the router begins discarding all additional datagrams. That is, the router discards the "tail" of the sequence.

Tail-drop has an interesting effect on TCP. In the simple case where datagrams traveling through a router carry segments from a single TCP connection, the loss causes TCP to enter slow-start, which reduces throughput until TCP begins receiving ACKs and increases the congestion window. A more severe problem can occur, however, when the datagrams traveling through a router carry segments from many TCP connections because tail-drop can cause global synchronization. To see why, observe that datagrams are typically multiplexed, with successive datagrams each coming from a different source. Thus, a tail-drop policy makes it likely that the router will discard one segment from N connections rather than N segments from one connection. The simultaneous loss causes all N instances of TCP to enter slow-start at the same time.

13.22 Random Early Discard (RED)

How can a router avoid global synchronization? The answer lies in a clever scheme that avoids tail-drop whenever possible. Known as *Random Early Discard*, *Random Early Drop*, or *Random Early Detection*, the scheme is more frequently referred to by its acronym, *RED*. A router that implements RED uses two threshold

†Technically, variance in delay is referred to as *jitter*.

values to mark positions in the queue: T_{min} and T_{max}. The general operation of RED can be described by three rules that determine the disposition of each arriving datagram:

- If the queue currently contains fewer than T_{min} datagrams, add the new datagram to the queue.
- If the queue contains more than T_{max} datagrams, discard the new datagram.
- If the queue contains between T_{min} and T_{max} datagrams, randomly discard the datagram according to a probability, p.

The randomness of RED means that instead of waiting until the queue overflows and then driving many TCP connections into slow-start, a router slowly and randomly drops datagrams as congestion increases. We can summarize:

> *RED Policy For Routers: if the input queue is full when a datagram arrives, discard the datagram; if the input queue is not full but the size exceeds a minimum threshold, avoid synchronization by discarding the datagram with probability* p.

The key to making RED work well lies in the choice of the thresholds T_{min} and T_{max}, and the discard probability p. T_{min} must be large enough to ensure that the output link has high utilization. Furthermore, because RED operates like tail-drop when the queue size exceeds T_{max}, the value must be greater than T_{min} by more than the typical increase in queue size during one TCP round trip time (e.g., set T_{max} at least twice as large as T_{min}). Otherwise, RED can cause the same global oscillations as tail-drop.

Computation of the discard probability, p, is the most complex aspect of RED. Instead of using a constant, a new value of p is computed for each datagram; the value depends on the relationship between the current queue size and the thresholds. To understand the scheme, observe that all RED processing can be viewed probabilistically. When the queue size is less than T_{min}, RED does not discard any datagrams, making the discard probability *0*. Similarly, when the queue size is greater than T_{max}, RED discards all datagrams, making the discard probability *1*. For intermediate values of queue size, (i.e., those between T_{min} and T_{max}), the probability can vary from *0* to *1* linearly.

Although the linear scheme forms the basis of RED's probability computation, a change must be made to avoid overreacting. The need for the change arises because network traffic is bursty, which results in rapid fluctuations of a router's queue. If RED used a simplistic linear scheme, later datagrams in each burst would be assigned high probability of being dropped (because they arrive when the queue has more entries). However, a router should not drop datagrams unnecessarily because doing so has a negative impact on TCP throughput. Thus, if a burst is short, it is unwise to drop datagrams because the queue will never overflow. Of course, RED cannot postpone discard indefinitely because a long-term burst will overflow the queue, resulting in a tail-drop policy which has the potential to cause global synchronization problems.

How can RED assign a higher discard probability as the queue fills without discarding datagrams from each burst? The answer lies in a technique borrowed from TCP: instead of using the actual queue size at any instant, RED computes a weighted average queue size, *avg*, and uses the average size to determine the probability. The value of *avg* is an exponential weighted average, updated each time a datagram arrives according to the equation:

$$avg = (1 - \gamma) * Old\_avg + \gamma * Current\_queue\_size$$

where γ denotes a value between *0* and *1*. If γ is small enough, the average will track long term trends, but will remain immune to short bursts†

In addition to equations that determine γ, RED contains other details that we have glossed over. For example, RED computations can be made extremely efficient by choosing constants as powers of two and using integer arithmetic. Another important detail concerns the measurement of queue size, which affects both the RED computation and its overall effect on TCP. In particular, because the time required to forward a datagram is proportional to its size, it makes sense to measure the queue in octets rather than in datagrams; doing so requires only minor changes to the equations for p and γ. Measuring queue size in octets affects the type of traffic dropped because it makes the discard probability proportional to the amount of data a sender puts in the stream rather than the number of segments. Small datagrams (e.g., those that carry remote login traffic or requests to servers) have lower probability of being dropped than large datagrams (e.g., those that carry file transfer traffic). One positive consequence of using size is that when acknowledgements travel over a congested path, they have a lower probability of being dropped. As a result, if a (large) data segment does arrive, the sending TCP will receive the ACK and will avoid unnecessary retransmission.

Both analysis and simulations show that RED works well. It handles congestion, avoids the synchronization that results from tail drop, and allows short bursts without dropping datagrams unnecessarily. The IETF now recommends that routers implement RED.

13.23 Establishing A TCP Connection

To establish a connection, TCP uses a *three-way handshake*. In the simplest case, the handshake proceeds as Figure 13.13 shows.

†An example value suggested for γ is .002.

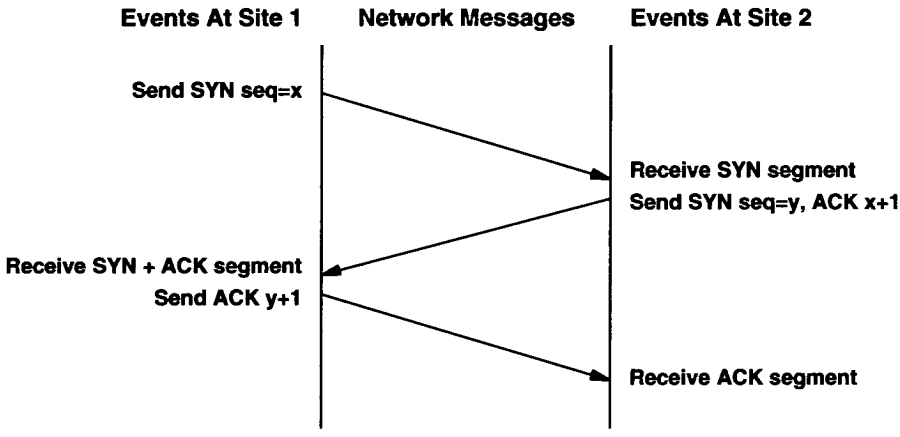

Figure 13.13 The sequence of messages in a three-way handshake. Time proceeds down the page; diagonal lines represent segments sent between sites. SYN segments carry initial sequence number information.

The first segment of a handshake can be identified because it has the SYN† bit set in the code field. The second message has both the SYN bit and ACK bits set, indicating that it acknowledges the first SYN segment as well as continuing the handshake. The final handshake message is only an acknowledgement and is merely used to inform the destination that both sides agree that a connection has been established.

Usually, the TCP software on one machine waits passively for the handshake, and the TCP software on another machine initiates it. However, the handshake is carefully designed to work even if both machines attempt to initiate a connection simultaneously. Thus, a connection can be established from either end or from both ends simultaneously. Once the connection has been established, data can flow in both directions equally well. There is no master or slave.

The three-way handshake is both necessary and sufficient for correct synchronization between the two ends of the connection. To understand why, remember that TCP builds on an unreliable packet delivery service, so messages can be lost, delayed, duplicated, or delivered out of order. Thus, the protocol must use a timeout mechanism and retransmit lost requests. Trouble arises if retransmitted and original requests arrive while the connection is being established, or if retransmitted requests are delayed until after a connection has been established, used, and terminated. A three-way handshake (plus the rule that TCP ignores additional requests for connection after a connection has been established) solves these problems.

†SYN stands for *synchronization*; it is pronounced "sin."

13.24 Initial Sequence Numbers

The three-way handshake accomplishes two important functions. It guarantees that both sides are ready to transfer data (and that they know they are both ready), and it allows both sides to agree on initial sequence numbers. Sequence numbers are sent and acknowledged during the handshake. Each machine must choose an initial sequence number at random that it will use to identify bytes in the stream it is sending. Sequence numbers cannot always start at the same value. In particular, TCP cannot merely choose sequence *1* every time it creates a connection (one of the exercises examines problems that can arise if it does). Of course, it is important that both sides agree on an initial number, so octet numbers used in acknowledgements agree with those used in data segments.

To see how machines can agree on sequence numbers for two streams after only three messages, recall that each segment contains both a sequence number field and an acknowledgement field. The machine that initiates a handshake, call it *A*, passes its initial sequence number, *x*, in the sequence field of the first SYN segment in the three-way handshake. The second machine, *B*, receives the SYN, records the sequence number, and replies by sending its initial sequence number in the sequence field as well as an acknowledgement that specifies *B* expects octet *x+1*. In the final message of the handshake, *A* "acknowledges" receiving from *B* all octets through *y*. In all cases, acknowledgements follow the convention of using the number of the *next* octet expected.

We have described how TCP usually carries out the three-way handshake by exchanging segments that contain a minimum amount of information. Because of the protocol design, it is possible to send data along with the initial sequence numbers in the handshake segments. In such cases, the TCP software must hold the data until the handshake completes. Once a connection has been established, the TCP software can release data being held and deliver it to a waiting application program quickly. The reader is referred to the protocol specification for the details.

13.25 Closing a TCP Connection

Two programs that use TCP to communicate can terminate the conversation gracefully using the *close* operation. Internally, TCP uses a modified three-way handshake to close connections. Recall that TCP connections are full duplex and that we view them as containing two independent stream transfers, one going in each direction. When an application program tells TCP that it has no more data to send, TCP will close the connection *in one direction*. To close its half of a connection, the sending TCP finishes transmitting the remaining data, waits for the receiver to acknowledge it, and then sends a segment with the FIN bit set. The receiving TCP acknowledges the FIN segment and informs the application program on its end that no more data is available (e.g., using the operating system's end-of-file mechanism).

Once a connection has been closed in a given direction, TCP refuses to accept more data for that direction. Meanwhile, data can continue to flow in the opposite

direction until the sender closes it. Of course, acknowledgements continue to flow back to the sender even after a connection has been closed. When both directions have been closed, the TCP software at each endpoint deletes its record of the connection.

The details of closing a connection are even more subtle than suggested above because TCP uses a modified three-way handshake to close a connection. Figure 13.14 illustrates the procedure.

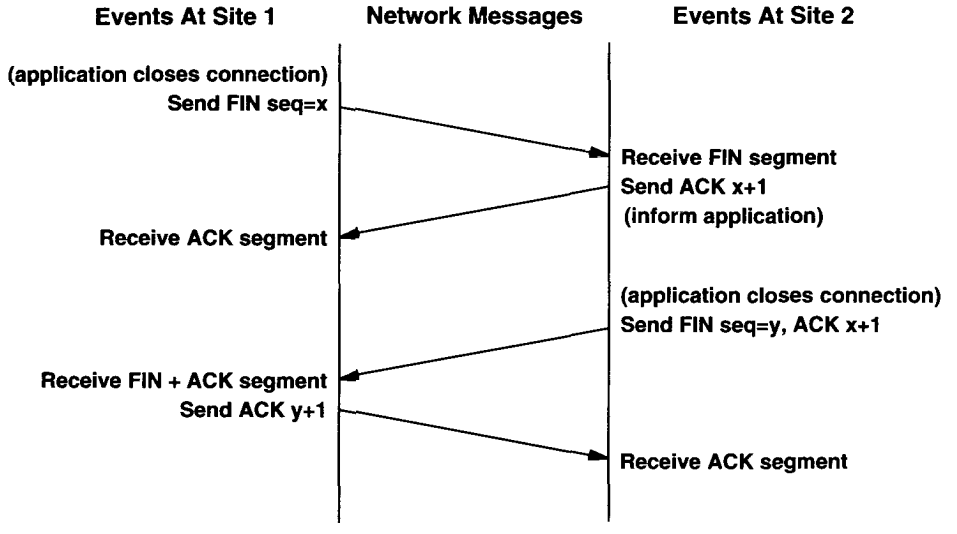

Figure 13.14 The modified three-way handshake used to close connections. The site that receives the first FIN segment acknowledges it immediately and then delays before sending the second FIN segment.

The difference between three-way handshakes used to establish and break connections occurs after a machine receives the initial FIN segment. Instead of generating a second FIN segment immediately, TCP sends an acknowledgement and then informs the application of the request to shut down. Informing the application program of the request and obtaining a response may take considerable time (e.g., it may involve human interaction). The acknowledgement prevents retransmission of the initial FIN segment during the wait. Finally, when the application program instructs TCP to shut down the connection completely, TCP sends the second FIN segment and the original site replies with the third message, an ACK.

13.26 TCP Connection Reset

Normally, an application program uses the close operation to shut down a connection when it finishes using it. Thus, closing connections is considered a normal part of use, analogous to closing files. Sometimes abnormal conditions arise that force an application program or the network software to break a connection. TCP provides a reset facility for such abnormal disconnections.

To reset a connection, one side initiates termination by sending a segment with the RST bit in the *CODE* field set. The other side responds to a reset segment immediately by aborting the connection. TCP also informs the application program that a reset occurred. A reset is an instantaneous abort that means that transfer in both directions ceases immediately, and resources such as buffers are released.

13.27 TCP State Machine

Like most protocols, the operation of TCP can best be explained with a theoretical model called a *finite state machine*. Figure 13.15 shows the TCP finite state machine, with circles representing states and arrows representing transitions between them. The label on each transition shows what TCP receives to cause the transition and what it sends in response. For example, the TCP software at each endpoint begins in the *CLOSED* state. Application programs must issue either a *passive open* command (to wait for a connection from another machine), or an *active open* command (to initiate a connection). An active open command forces a transition from the *CLOSED* state to the *SYN SENT* state. When TCP follows the transition, it emits a SYN segment. When the other end returns a segment that contains a SYN plus ACK, TCP moves to the *ESTABLISHED* state and begins data transfer.

The *TIMED WAIT* state reveals how TCP handles some of the problems incurred with unreliable delivery. TCP keeps a notion of *maximum segment lifetime* (*MSL*), the maximum time an old segment can remain alive in an internet. To avoid having segments from a previous connection interfere with a current one, TCP moves to the *TIMED WAIT* state after closing a connection. It remains in that state for twice the maximum segment lifetime before deleting its record of the connection. If any duplicate segments happen to arrive for the connection during the timeout interval, TCP will reject them. However, to handle cases where the last acknowledgement was lost, TCP acknowledges valid segments and restarts the timer. Because the timer allows TCP to distinguish old connections from new ones, it prevents TCP from responding with a *RST* (reset) if the other end retransmits a *FIN* request.

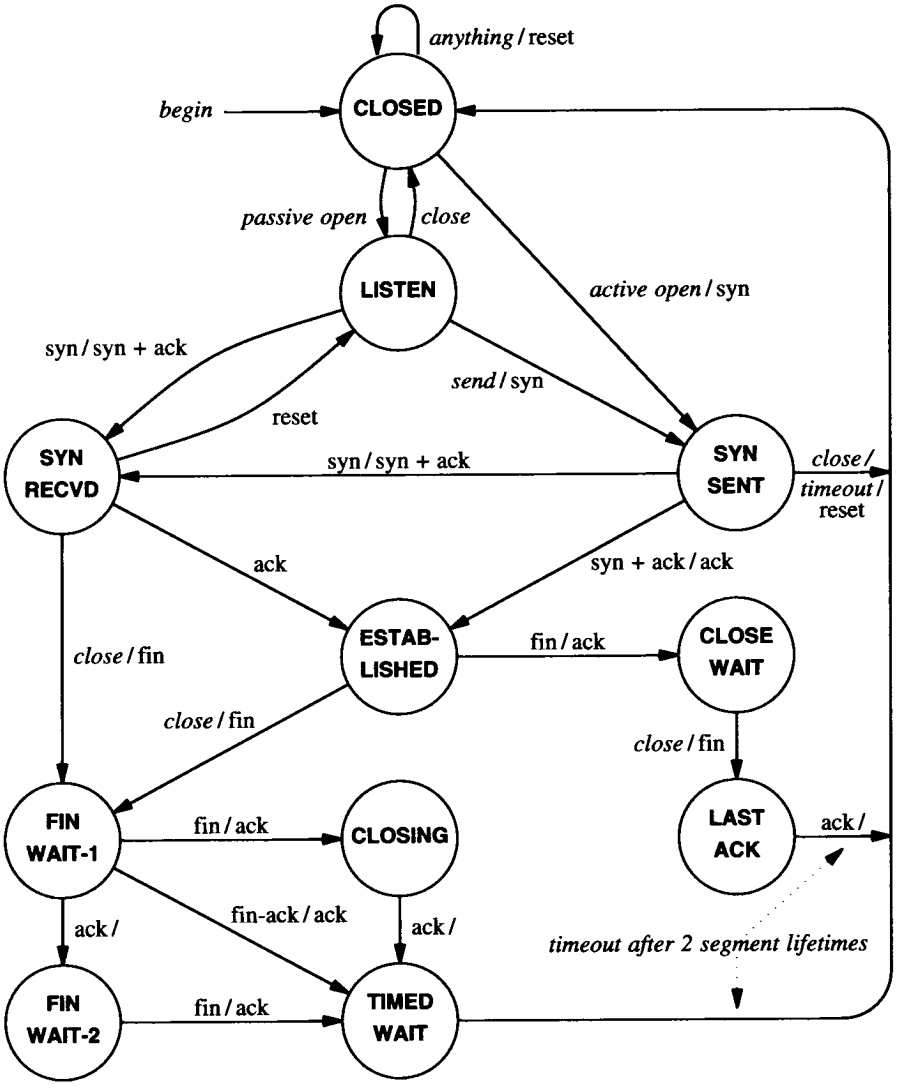

Figure 13.15 The TCP finite state machine. Each endpoint begins in the *closed* state. Labels on transitions show the input that caused the transition followed by the output if any.

13.28 Forcing Data Delivery

We have said that TCP is free to divide the stream of data into segments for transmission without regard to the size of transfer that application programs use. The chief advantage of allowing TCP to choose a division is efficiency. It can accumulate enough octets in a buffer to make segments reasonably long, reducing the high overhead that occurs when segments contain only a few data octets.

Although buffering improves network throughput, it can interfere with some applications. Consider using a TCP connection to pass characters from an interactive terminal to a remote machine. The user expects instant response to each keystroke. If the sending TCP buffers the data, response may be delayed, perhaps for hundreds of keystrokes. Similarly, because the receiving TCP may buffer data before making it available to the application program on its end, forcing the sender to transmit data may not be sufficient to guarantee delivery.

To accommodate interactive users, TCP provides a *push* operation that an application program can use to force delivery of octets currently in the stream without waiting for the buffer to fill. The push operation does more than force TCP to send a segment. It also requests TCP to set the *PSH* bit in the segment code field, so the data will be delivered to the application program on the receiving end. Thus, when sending data from an interactive terminal, the application uses the push function after each keystroke. Similarly, application programs can force output to be sent and displayed on the terminal promptly by calling the push function after writing a character or line.

13.29 Reserved TCP Port Numbers

Like UDP, TCP combines static and dynamic port binding, using a set of *well-known port assignments* for commonly invoked programs (e.g., electronic mail), but leaving most port numbers available for the operating system to allocate as programs need them. Although the standard originally reserved port numbers less than *256* for use as well-known ports, numbers over 1024 have now been assigned. Figure 13.16 lists some of the currently assigned TCP ports. It should be pointed out that although TCP and UDP port numbers are independent, the designers have chosen to use the same integer port numbers for any service that is accessible from both UDP and TCP. For example, a domain name server can be accessed either with TCP or with UDP. In either protocol, port number *53* has been reserved for servers in the domain name system.

13.30 TCP Performance

As we have seen, TCP is a complex protocol that handles communication over a wide variety of underlying network technologies. Many people assume that because TCP tackles a much more complex task than other transport protocols, the code must be cumbersome and inefficient. Surprisingly, the generality we discussed does not seem to

hinder TCP performance. Experiments at Berkeley have shown that the same TCP that operates efficiently over the global Internet can deliver 8 Mbps of sustained throughput of user data between two workstations on a 10 Mbps Ethernet†. At Cray Research, Inc., researchers have demonstrated TCP throughput approaching a gigabit per second.

Decimal	Keyword	UNIX Keyword	Description
0			Reserved
1	TCPMUX	-	TCP Multiplexor
7	ECHO	echo	Echo
9	DISCARD	discard	Discard
11	USERS	systat	Active Users
13	DAYTIME	daytime	Daytime
15	-	netstat	Network status program
17	QUOTE	qotd	Quote of the Day
19	CHARGEN	chargen	Character Generator
20	FTP-DATA	ftp-data	File Transfer Protocol (data)
21	FTP	ftp	File Transfer Protocol
22	SSH	ssh	Secure Shell
23	TELNET	telnet	Terminal Connection
25	SMTP	smtp	Simple Mail Transport Protocol
37	TIME	time	Time
43	NICNAME	whois	Who Is
53	DOMAIN	nameserver	Domain Name Server
67	BOOTPS	bootps	BOOTP or DHCP Server
77	-	rje	any private RJE service
79	FINGER	finger	Finger
80	WWW	www	World Wide Web Server
88	KERBEROS	kerberos	Kerberos Security Service
95	SUPDUP	supdup	SUPDUP Protocol
101	HOSTNAME	hostnames	NIC Host Name Server
102	ISO-TSAP	iso-tsap	ISO-TSAP
103	X400	x400	X.400 Mail Service
104	X400-SND	x400-snd	X.400 Mail Sending
110	POP3	pop3	Post Office Protocol Vers. 3
111	SUNRPC	sunrpc	SUN Remote Procedure Call
113	AUTH	auth	Authentication Service
117	UUCP-PATH	uucp-path	UUCP Path Service
119	NNTP	nntp	USENET News Transfer Protocol
123	NTP	ntp	Network Time Protocol
139	NETBIOS-SSN	-	NETBIOS Session Service
161	SNMP	snmp	Simple Network Management Proto

Figure 13.16 Examples of currently assigned TCP port numbers. To the extent possible, protocols like UDP use the same numbers.

†Ethernet, IP, and TCP headers and the required inter-packet gap account for the remaining bandwidth.

13.31 Silly Window Syndrome And Small Packets

Researchers who helped developed TCP observed a serious performance problem that can result when the sending and receiving applications operate at different speeds. To understand the problem, remember that TCP buffers incoming data, and consider what can happen if a receiving application chooses to read incoming data one octet at a time. When a connection is first established, the receiving TCP allocates a buffer of *K* bytes, and uses the *WINDOW* field in acknowledgement segments to advertise the available buffer size to the sender. If the sending application generates data quickly, the sending TCP will transmit segments with data for the entire window. Eventually, the sender will receive an acknowledgement that specifies the entire window has been filled, and no additional space remains in the receiver's buffer.

When the receiving application reads an octet of data from a full buffer, one octet of space becomes available. We said that when space becomes available in its buffer, TCP on the receiving machine generates an acknowledgement that uses the *WINDOW* field to inform the sender. In the example, the receiver will advertise a window of *1* octet. When it learns that space is available, the sending TCP responds by transmitting a segment that contains one octet of data.

Although single-octet window advertisements work correctly to keep the receiver's buffer filled, they result in a series of small data segments. The sending TCP must compose a segment that contains one octet of data, place the segment in an IP datagram, and transmit the result. When the receiving application reads another octet, TCP generates another acknowledgement, which causes the sender to transmit another segment that contains one octet of data. The resulting interaction can reach a steady state in which TCP sends a separate segment for each octet of data.

Transferring small segments consumes unnecessary network bandwidth and introduces unnecessary computational overhead. The transmission of small segments consumes unnecessary network bandwidth because each datagram carries only one octet of data; the ratio of header to data is large. Computational overhead arises because TCP on both the sending and receiving computers must process each segment. The sending TCP software must allocate buffer space, form a segment header, and compute a checksum for the segment. Similarly, IP software on the sending machine must encapsulate the segment in a datagram, compute a header checksum, route the datagram, and transfer it to the appropriate network interface. On the receiving machine, IP must verify the IP header checksum and pass the segment to TCP. TCP must verify the segment checksum, examine the sequence number, extract the data, and place it in a buffer.

Although we have described how small segments result when a receiver advertises a small available window, a sender can also cause each segment to contain a small amount of data. For example, imagine a TCP implementation that aggressively sends data whenever it is available, and consider what happens if a sending application generates data one octet at a time. After the application generates an octet of data, TCP creates and transmits a segment. TCP can also send a small segment if an application generates data in fixed-sized blocks of *B* octets, and the sending TCP extracts data from

the buffer in maximum segment sized blocks, M, where $M \neq B$, because the last block in a buffer can be small.

Known as *silly window syndrome* (*SWS*), the problem plagued early TCP implementations. To summarize,

> *Early TCP implementations exhibited a problem known as* silly window syndrome *in which each acknowledgement advertises a small amount of space available and each segment carries a small amount of data.*

13.32 Avoiding Silly Window Syndrome

TCP specifications now include heuristics that prevent silly window syndrome. A heuristic used on the sending machine avoids transmitting a small amount of data in each segment. Another heuristic used on the receiving machine avoids sending small increments in window advertisements that can trigger small data packets. Although the heuristics work well together, having both the sender and receiver avoid silly window helps ensure good performance in the case that one end of a connection fails to correctly implement silly window avoidance.

In practice, TCP software must contain both sender and receiver silly window avoidance code. To understand why, recall that a TCP connection is full duplex — data can flow in either direction. Thus, an implementation of TCP includes code to send data as well as code to receive it.

13.32.1 Receive-Side Silly Window Avoidance

The heuristic a receiver uses to avoid silly window is straightforward and easiest to understand. In general, a receiver maintains an internal record of the currently available window, but delays advertising an increase in window size to the sender until the window can advance a significant amount. The definition of "significant" depends on the receiver's buffer size and the maximum segment size. TCP defines it to be the minimum of one half of the receiver's buffer or the number of data octets in a maximum-sized segment.

Receive-side silly window prevents small window advertisements in the case where a receiving application extracts data octets slowly. For example, when a receiver's buffer fills completely, it sends an acknowledgement that contains a zero window advertisement. As the receiving application extracts octets from the buffer, the receiving TCP computes the newly available space in the buffer. Instead of sending a window advertisement immediately, however, the receiver waits until the available space reaches one half of the total buffer size or a maximum sized segment. Thus, the sender always receives large increments in the current window, allowing it to transfer large segments. The heuristic can be summarized as follows.

Receive-Side Silly Window Avoidance: Before sending an updated window advertisement after advertising a zero window, wait for space to become available that is either at least 50% of the total buffer size or equal to a maximum sized segment.

13.32.2 Delayed Acknowledgements

Two approaches have been taken to implement silly window avoidance on the receive side. In the first approach, TCP acknowledges each segment that arrives, but does not advertise an increase in its window until the window reaches the limits specified by the silly window avoidance heuristic. In the second approach, TCP delays sending an acknowledgement when silly window avoidance specifies that the window is not sufficiently large to advertise. The standards recommend delaying acknowledgements.

Delayed acknowledgements have both advantages and disadvantages. The chief advantage arises because delayed acknowledgements can decrease traffic and thereby increase throughput. For example, if additional data arrives during the delay period, a single acknowledgement will acknowledge all data received. If the receiving application generates a response immediately after data arrives (e.g., character echo in a remote login session), a short delay may permit the acknowledgement to piggyback on a data segment. Furthermore, TCP cannot move its window until the receiving application extracts data from the buffer. In cases where the receiving application reads data as soon as it arrives, a short delay allows TCP to send a single segment that acknowledges the data and advertises an updated window. Without delayed acknowledgements, TCP will acknowledge the arrival of data immediately, and later send an additional acknowledgement to update the window size.

The disadvantages of delayed acknowledgements should be clear. Most important, if a receiver delays acknowledgements too long, the sending TCP will retransmit the segment. Unnecessary retransmissions lower throughput because they waste network bandwidth. In addition, retransmissions require computational overhead on the sending and receiving machines. Furthermore, TCP uses the arrival of acknowledgements to estimate round trip times; delaying acknowledgements can confuse the estimate and make retransmission times too long.

To avoid potential problems, the TCP standards place a limit on the time TCP delays an acknowledgement. Implementations cannot delay an acknowledgement for more than 500 milliseconds. Furthermore, to guarantee that TCP receives a sufficient number of round trip estimates, the standard recommends that a receiver should acknowledge at least every other data segment.

13.32.3 Send-Side Silly Window Avoidance

The heuristic a sending TCP uses to avoid silly window syndrome is both surprising and elegant. Recall that the goal is to avoid sending small segments. Also recall that a sending application can generate data in arbitrarily small blocks (e.g., one octet at a time). Thus, to achieve the goal, a sending TCP must allow the sending application to make multiple calls to *write*, and must collect the data transferred in each call before transmitting it in a single, large segment. That is, a sending TCP must delay sending a segment until it can accumulate a reasonable amount of data. The technique is known as *clumping*.

The question arises, "How long should TCP wait before transmitting data?" On one hand, if TCP waits too long, the application experiences large delays. More important, TCP cannot know whether to wait because it cannot know whether the application will generate more data in the near future. On the other hand, if TCP does not wait long enough, segments will be small and throughput will be low.

Protocols designed prior to TCP confronted the same problem and used techniques to clump data into larger packets. For example, to achieve efficient transfer across a network, early remote terminal protocols delayed transmitting each keystroke for a few hundred milliseconds to determine whether the user would continue to press keys. Because TCP is designed to be general, however, it can be used by a diverse set of applications. Characters may travel across a TCP connection because a user is typing on a keyboard or because a program is transferring a file. A fixed delay is not optimal for all applications.

Like the algorithm TCP uses for retransmission and the slow-start algorithm used to avoid congestion, the technique a sending TCP uses to avoid sending small packets is adaptive — the delay depends on the current performance of the internet. Like slow-start, send-side silly window avoidance is called *self clocking* because it does not compute delays. Instead, TCP uses the arrival of an acknowledgement to trigger the transmission of additional packets. The heuristic can be summarized:

> *Send-Side Silly Window Avoidance: When a sending application generates additional data to be sent over a connection for which previous data has been transmitted but not acknowledged, place the new data in the output buffer as usual, but do not send additional segments until there is sufficient data to fill a maximum-sized segment. If still waiting to send when an acknowledgement arrives, send all data that has accumulated in the buffer. Apply the rule even when the user requests a* push *operation.*

If an application generates data one octet at a time, TCP will send the first octet immediately. However, until the ACK arrives, TCP will accumulate additional octets in its buffer. Thus, if the application is reasonably fast compared to the network (i.e., a file transfer), successive segments will each contain many octets. If the application is slow compared to the network (e.g., a user typing on a keyboard), small segments will be sent without long delay.

Known as the *Nagle algorithm* after its inventor, the technique is especially elegant because it requires little computational overhead. A host does not need to keep separate timers for each connection, nor does the host need to examine a clock when an application generates data. More important, although the technique adapts to arbitrary combinations of network delay, maximum segment size, and application speed, it does not lower throughput in conventional cases.

To understand why throughput remains high for conventional communication, observe that applications optimized for high throughput do not generate data one octet at a time (doing so would incur unnecessary operating system overhead). Instead, such applications write large blocks of data with each call. Thus, the outgoing TCP buffer begins with sufficient data for at least one maximum size segment. Furthermore, because the application produces data faster than TCP can transfer data, the sending buffer remains nearly full, and TCP does not delay transmission. As a result, TCP continues to send segments at whatever rate the internet can tolerate, while the application continues to fill the buffer. To summarize:

> *TCP now requires the sender and receiver to implement heuristics that avoid the silly window syndrome. A receiver avoids advertising a small window, and a sender uses an adaptive scheme to delay transmission so it can clump data into large segments.*

13.33 Summary

The Transmission Control Protocol, TCP, defines a key service provided by an internet, namely, reliable stream delivery. TCP provides a full duplex connection between two machines, allowing them to exchange large volumes of data efficiently.

Because it uses a sliding window protocol, TCP can make efficient use of a network. Because it makes few assumptions about the underlying delivery system, TCP is flexible enough to operate over a large variety of delivery systems. Because it provides flow control, TCP allows systems of widely varying speeds to communicate.

The basic unit of transfer used by TCP is a segment. Segments are used to pass data or control information (e.g., to allow TCP software on two machines to establish connections or break them). The segment format permits a machine to piggyback acknowledgements for data flowing in one direction by including them in the segment headers of data flowing in the opposite direction.

TCP implements flow control by having the receiver advertise the amount of data it is willing to accept. It also supports out-of-band messages using an urgent data facility and forces delivery using a push mechanism.

The current TCP standard specifies exponential backoff for retransmission timers and congestion avoidance algorithms like slow-start, multiplicative decrease, and additive increase. In addition, TCP uses heuristics to avoid transferring small packets. Finally, the IETF recommends that routers use RED instead of tail-drop because doing so avoids TCP synchronization and improves throughput.

FOR FURTHER STUDY

The standard for TCP can be found in Postel [RFC 793]; Braden [RFC 1122] contains an update that clarifies several points. Clark [RFC 813] describes TCP window management, Clark [RFC 816] describes fault isolation and recovery, and Postel [RFC 879] reports on TCP maximum segment sizes. Nagle [RFC 896] comments on congestion in TCP/IP networks and explains the effect of self clocking for send-side silly window avoidance. Karn and Partridge [1987] discusses estimation of round-trip times, and presents Karn's algorithm. Jacobson [1988] gives the congestion control algorithms that are now a required part of the standard. Floyd and Jacobson [1993] presents the RED scheme, and Clark and Fang [1998] discusses an allocation framework that uses RED. Tomlinson [1975] considers the three-way handshake in more detail. Mills [RFC 889] reports measurements of Internet round-trip delays. Jain [1986] describes timer-based congestion control in a sliding window environment. Borman [April 1989] summarizes experiments with high-speed TCP on Cray computers.

EXERCISES

13.1 TCP uses a finite field to contain stream sequence numbers. Study the protocol specification to find out how it allows an arbitrary length stream to pass from one machine to another.

13.2 The text notes that one of the TCP options permits a receiver to specify the maximum segment size it is willing to accept. Why does TCP support an option to specify maximum segment size when it also has a window advertisement mechanism?

13.3 Under what conditions of delay, bandwidth, load, and packet loss will TCP retransmit significant volumes of data unnecessarily?

13.4 Lost TCP acknowledgements do not necessarily force retransmissions. Explain why.

13.5 Experiment with local machines to determine how TCP handles machine restart. Establish a connection (e.g., a remote login) and leave it idle. Wait for the destination machine to crash and restart, and then force the local machine to send a TCP segment (e.g., by typing characters to the remote login).

13.6 Imagine an implementation of TCP that discards segments that arrive out of order, even if they fall in the current window. That is, the imagined version only accepts segments that extend the byte stream it has already received. Does it work? How does it compare to a standard TCP implementation?

13.7 Consider computation of a TCP checksum. Assume that although the checksum field in the segment has *not* been set to zero, the result of computing the checksum *is* zero. What can you conclude?

13.8 What are the arguments for and against automatically closing idle connections?

13.9 If two application programs use TCP to send data but only send one character per segment (e.g., by using the PUSH operation), what is the maximum percent of the network bandwidth they will have for their data?

13.10 Suppose an implementation of TCP uses initial sequence number *1* when it creates a connection. Explain how a system crash and restart can confuse a remote system into believing that the old connection remained open.

13.11 Look at the round-trip time estimation algorithm suggested in the ISO TP-4 protocol specification and compare it to the TCP algorithm discussed in this chapter. Which would you prefer to use?

13.12 Find out how implementations of TCP must solve the *overlapping segment problem.* The problem arises because the receiver must accept only one copy of all bytes from the data stream even if the sender transmits two segments that partially overlap one another (e.g., the first segment carries bytes 100 through 200 and the second carries bytes 150 through 250).

13.13 Trace the TCP finite state machine transitions for two sites that execute a passive and active open and step through the three-way handshake.

13.14 Read the TCP specification to find out the exact conditions under which TCP can make the transition from *FIN WAIT-1* to *TIMED WAIT.*

13.15 Trace the TCP state transitions for two machines that agree to close a connection gracefully.

13.16 Assume TCP is sending segments using a maximum window size (64 Kbytes) on a channel that has infinite bandwidth and an average roundtrip time of 20 milliseconds. What is the maximum throughput? How does throughput change if the roundtrip time increases to 40 milliseconds (while bandwidth remains infinite)?

13.17 As the previous exercise illustrates, higher throughput can be achieved with larger windows. One of the drawbacks of the TCP segment format is the size of the field devoted to window advertisement. How can TCP be extended to allow larger windows without changing the segment format?

13.18 Can you derive an equation that expresses the maximum possible TCP throughput as a function of the network bandwidth, the network delay, and the time to process a segment and generate an acknowledgement. Hint: consider the previous exercise.

13.19 Describe (abnormal) circumstances that can leave one end of a connection in state *FIN WAIT-2* indefinitely (hint: think of datagram loss and system crashes).

13.20 Show that when a router implements RED, the probability a packet will be discarded from a particular TCP connection is proportional to the percentage of traffic that the connection generates.

14

Routing: Cores, Peers, And Algorithms

14.1 Introduction

Previous chapters concentrate on the network level services TCP/IP offers and the details of the protocols in hosts and routers that provide those services. In the discussion, we assumed that routers always contain correct routes, and we observed that routers can ask directly connected hosts to change routes with the ICMP redirect mechanism.

This chapter considers two broad questions: "What values should routing tables contain?" and "How can those values be obtained?" To answer the first question, we will consider the relationship between internet architecture and routing. In particular, we will discuss internets structured around a backbone and those composed of multiple peer networks, and consider the consequences for routing. While many of our examples are drawn from the global Internet, the ideas apply equally well to smaller corporate internets. To answer the second question, we will consider the two basic types of route propagation algorithms and see how each supplies routing information automatically.

We begin by discussing routing in general. Later sections concentrate on internet architecture and describe the algorithms routers use to exchange routing information. Chapters 15 and 16 continue to expand our discussion of routing. They explore protocols that routers owned by two independent administrative groups use to exchange information, and protocols that a single group uses among all its routers.

14.2 The Origin Of Routing Tables

Recall from Chapter 3 that IP routers provide active interconnections among networks. Each router attaches to two or more physical networks and forwards IP datagrams among them, accepting datagrams that arrive over one network interface, and routing them out over another interface. Except for destinations on directly attached networks, hosts pass all IP traffic to routers which forward datagrams on toward their final destinations. A datagram travels from router to router until it reaches a router that attaches directly to the same network as the final destination. Thus, the router system forms the architectural basis of an internet and handles all traffic except for direct delivery from one host to another.

Chapter 8 describes the IP routing algorithm that hosts and routers follow to forward datagrams, and shows how the algorithm uses a table to make routing decisions. Each entry in the routing table specifies the network portion of a destination address and gives the address of the next machine along a path used to reach that network. Like hosts, routers directly deliver datagrams to destinations on networks to which the router attaches.

Although we have seen the basics of datagram forwarding, we have not said how hosts or routers obtain the information for their routing tables. The issue has two aspects: *what* values should be placed in the tables, and *how* routers obtain those values. Both choices depend on the architectural complexity and size of the internet as well as administrative policies.

In general, establishing routes involves initialization and update. Each router must establish an initial set of routes when it starts, and it must update the table as routes change (e.g., when a network interface fails). Initialization depends on the operating system. In some systems, the router reads an initial routing table from secondary storage at startup, keeping it resident in main memory. In others, the operating system begins with an empty table which must be filled in by executing explicit commands (e.g., commands found in a startup command script). Finally, some operating systems start by deducing an initial set of routes from the set of addresses for the local networks to which the machine attaches and contacting a neighboring machine to ask for additional routes.

Once an initial routing table has been built, a router must accommodate changes in routes. In small, slowly changing internets, managers can establish and modify routes by hand. In large, rapidly changing environments, however, manual update is impossibly slow and prone to human errors. Automated methods are needed.

Before we can understand the automatic routing table update protocols used in IP routers, we need to review several underlying ideas. The next sections do so, providing the necessary conceptual foundation for routing. Later sections discuss internet architecture and the protocols routers use to exchange routing information.

14.3 Routing With Partial Information

The principal difference between routers and typical hosts is that hosts usually know little about the structure of the internet to which they connect. Hosts do not have complete knowledge of all possible destination addresses, or even of all possible destination networks. In fact, many hosts have only two routes in their routing table: a route for the local network and a default route for a nearby router. The host sends all nonlocal datagrams to the local router for delivery. The point is that:

> *A host can route datagrams successfully even if it only has partial routing information because it can rely on a router.*

Can routers also route datagrams with only partial information? Yes, but only under certain circumstances. To understand the criteria, imagine an internet to be a foreign country crisscrossed with dirt roads that have directional signs posted at intersections. Imagine that you have no map, cannot ask directions because you cannot speak the local language, have no ideas about visible landmarks, but you need to travel to a village named *Sussex*. You leave on your journey, following the only road out of town and begin to look for directional signs. The first sign reads:

Norfolk to the left; Hammond to the right; others straight ahead.†

Because the destination you seek is not listed explicitly, you continue straight ahead. In routing jargon, we say you follow a *default route*. After several more signs, you finally find one that reads:

Essex to the left; Sussex to the right; others straight ahead.

You turn to the right, follow several more signs, and emerge on a road that leads to Sussex.

Our imagined travel is analogous to a datagram traversing an internet, and the road signs are analogous to routing tables in routers along the path. Without a map or other navigational aids, travel is completely dependent on road signs, just as datagram routing in an internet depends entirely on routing tables. Clearly, it is possible to navigate even though each road sign contains only partial information.

A central question concerns correctness. As a traveler, you might ask, "How can I be sure that following the signs will lead to my final destination?" You also might ask, "How can I be sure that following the signs will lead me to my destination along a shortest path?" These questions may seem especially troublesome if you pass many signs without finding your destination listed explicitly. Of course, the answers depend on the topology of the road system and the contents of the signs, but the fundamental idea is that when taken as a whole, the information on the signs should be both consistent and complete. Looking at this another way, we see that it is not necessary for each intersection to have a sign for every destination. The signs can list default paths as

†Fortunately, signs are printed in a language you can read.

long as all explicit signs point along a shortest path, and the turns for shortest paths to all destinations are marked. A few examples will explain some ways that consistency can be achieved.

At one extreme, consider a simple star-shaped topology of roads in which each village has exactly one road leading to it, and all those roads meet at a central point. To guarantee consistency, the sign at the central intersection must contain information about all possible destinations. At the other extreme, imagine an arbitrary set of roads with signs at all intersections listing all possible destinations. To guarantee consistency, it must be true that at any intersection if the sign for destination D points to road R, no road other than R leads to a shorter path to D.

Neither of these architectural extremes works well for an internet router system. On one hand, the central intersection approach fails because no machine is fast enough to serve as a central switch through which all traffic passes. On the other hand, having information about all possible destinations in all routers is impractical because it requires propagating large volumes of information whenever a change occurs or whenever administrators need to check consistency. Thus, we seek a solution that allows groups to manage local routers autonomously, adding new network interconnections and routes without changing distant routers.

To help explain some of the architecture described later, consider a third topology in which half the cities lie in the eastern part of the country and half lie in the western part. Suppose a single bridge spans the river that separates east from west. Assume that people living in the eastern part do not like westerners, so they are willing to allow road signs that list destinations in the east but none in the west. Assume that people living in the west do the opposite. Routing will be consistent if every road sign in the east lists all eastern destinations explicitly and points the default path to the bridge, while every road sign in the west lists all western destinations explicitly and points the default path to the bridge.

14.4 Original Internet Architecture And Cores

Much of our knowledge of routing and route propagation protocols has been derived from experience with the global Internet. When TCP/IP was first developed, participating research sites were connected to the ARPANET, which served as the Internet backbone. During initial experiments, each site managed routing tables and installed routes to other destinations by hand. As the fledgling Internet began to grow, it became apparent that manual maintenance of routes was impractical; automated mechanisms were needed.

The Internet designers selected a router architecture that consisted of a small, central set of routers that kept complete information about all possible destinations, and a larger set of outlying routers that kept partial information. In terms of our analogy, it is like designating a small set of centrally located intersections to have signs that list all destinations, and allowing the outlying intersections to list only local destinations. As long as the default route at each outlying intersection points to one of the central inter-

sections, travelers will eventually reach their destination. The advantage of using partial information in outlying routers is that it permits local administrators to manage local structural changes without affecting other parts of the Internet. The disadvantage is that it introduces the potential for inconsistency. In the worst case, an error in an outlying router can make distant routes unreachable.

We can summarize these ideas:

> *The routing table in a given router contains partial information about possible destinations. Routing that uses partial information allows sites autonomy in making local routing changes, but introduces the possibility of inconsistencies that may make some destinations unreachable from some sources.*

Inconsistencies among routing tables usually arise from errors in the algorithms that compute routing tables, incorrect data supplied to those algorithms, or from errors that occur while transmitting the results to other routers. Protocol designers look for ways to limit the impact of errors, with the objective being to keep all routes consistent at all times. If routes become inconsistent for some reason, the routing protocols should be robust enough to detect and correct the errors quickly. Most important, the protocols should be designed to constrain the effect of errors.

14.5 Core Routers

Loosely speaking, early Internet routers could be partitioned into two groups, a small set of *core routers* controlled by the Internet Network Operations Center (INOC), and a larger set of *noncore routers*† controlled by individual groups. The core system was designed to provide reliable, consistent, authoritative routes for all possible destinations; it was the glue that held the Internet together and made universal interconnection possible. By fiat, each site assigned an Internet network address had to arrange to advertise that address to the core system. The core routers communicated among themselves, so they could guarantee that the information they shared was consistent. Because a central authority monitored and controlled the core routers, they were highly reliable.

To fully understand the core router system, it is necessary to recall that the Internet evolved with a wide-area network, the ARPANET, already in place. When the Internet experiments began, designers thought of the ARPANET as a main backbone on which to build. Thus, a large part of the motivation for the core router system came from the desire to connect local networks to the ARPANET. Figure 14.1 illustrates the idea.

†The terms *stub* and *nonrouting* have also been used in place of *noncore*.

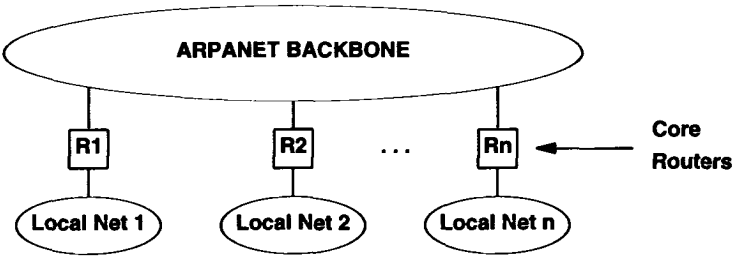

Figure 14.1 The early Internet core router system viewed as a set of routers that connect local area networks to the ARPANET. Hosts on the local networks pass all nonlocal traffic to the closest core router.

To understand why such an architecture does not lend itself to routing with partial information, suppose that a large internet consists entirely of local area networks, each attached to a backbone network through a router. Also imagine that some of the routers rely on default routes. Now consider the path a datagram follows. At the source site, the local router checks to see if it has an explicit route to the destination and, if not, sends the datagram along the path specified by its default route. All datagrams for which the router has no route follow the same default path regardless of their ultimate destination. The next router along the path diverts datagrams for which it has an explicit route, and sends the rest along its default route. To ensure global consistency, the chain of default routes must reach every router in a giant cycle as Figure 14.2 shows. Thus, the architecture requires all local sites to coordinate their default routes. In addition, depending on default routes can be inefficient even when it is consistent. As Figure 14.2 shows, in the worst case a datagram will pass through all n routers as it travels from source to destination instead of going directly across the backbone.

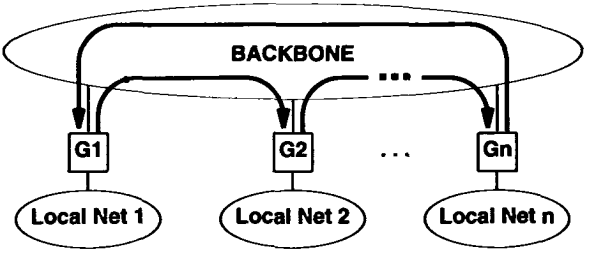

Figure 14.2 A set of routers connected to a backbone network with default routes shown. Routing is inefficient even though it is consistent.

To avoid the inefficiencies default routes cause, Internet designers arranged for all core routers to exchange routing information so that each would have complete information about optimal routes to all possible destinations. Because each core router knew routes to all possible destinations, it did not need a default route. If the destination address on a datagram was not in a core router's routing table, the router would generate an ICMP destination unreachable message and drop the datagram. In essence, the core design avoided inefficiency by eliminating default routes.

Figure 14.3 depicts the conceptual basis of a core routing architecture. The figure shows a central core system consisting of one or more core routers, and a set of outlying routers at local sites. Outlying routers keep information about local destinations and use a default route that sends datagrams destined for other sites to the core.

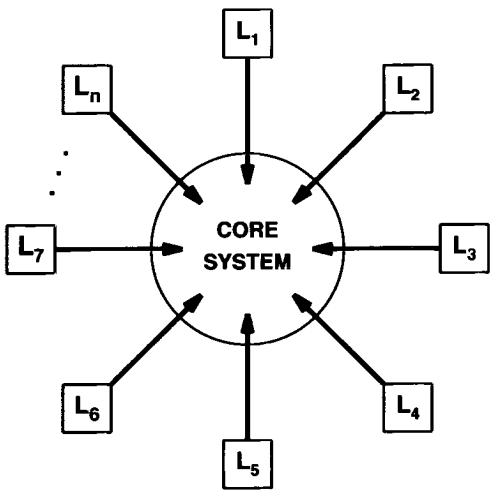

Figure 14.3 The routing architecture of a simplistic core system showing default routes. Core routers do not use default routes; outlying routers, labeled L_i, each have a default route that points to the core.

Although the simplistic core architecture illustrated in Figure 14.3 is easy to understand, it became impractical for three reasons. First, the Internet outgrew a single, centrally managed long-haul backbone. The topology became complex and the protocols needed to maintain consistency among core routers became nontrivial. Second, not every site could have a core router connected to the backbone, so additional routing structure and protocols were needed. Third, because core routers all interacted to ensure consistent routing information, the core architecture did not scale to arbitrary size. We will return to this last problem in Chapter 15 after we examine the protocols that the core system used to exchange routing information.

14.6 Beyond The Core Architecture To Peer Backbones

The introduction of the NSFNET backbone into the Internet added new complexity to the routing structure. From the core system point of view, the connection to NSFNET was initially no different than the connection to any other site. NSFNET attached to the ARPANET backbone through a single router in Pittsburgh. The core had explicit routes to all destinations in NSFNET. Routers inside NSFNET knew about local destinations and used a default route to send all non-NSFNET traffic to the core via the Pittsburgh router.

As NSFNET grew to become a major part of the Internet, it became apparent that the core routing architecture would not suffice. The most important conceptual change occurred when multiple connections were added between the ARPANET and NSFNET backbones. We say that the two became *peer backbone networks* or simply *peers*. Figure 14.4 illustrates the resulting peer topology.

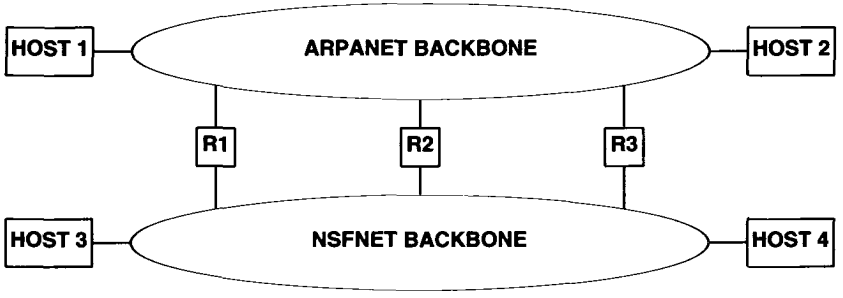

Figure 14.4 An example of peer backbones interconnected through multiple routers. The diagram illustrates the architecture of the Internet in 1989. In later generations, parallel backbones were each owned by an ISP.

To understand the difficulties of IP routing among peer backbones, consider routes from host *3* to host *2* in Figure 14.4. Assume for the moment that the figure shows geographic orientation, so host *3* is on the West Coast attached to the NSFNET backbone while host *2* is on the East Coast attached to the ARPANET backbone. When establishing routes between hosts *3* and *2*, the managers must decide whether to (a) route the traffic from host *3* through the West Coast router, *R1*, and then across the ARPANET backbone, or (b) route the traffic from host *3* across the NSFNET backbone, through the Midwest router, *R2*, and then across the ARPANET backbone to host *2*, or (c) route the traffic across the NSFNET backbone, through the East Coast router, *R3*, and then to host *2*. A more circuitous route is possible as well: traffic could flow from host *3* through the West Coast router, across the ARPANET backbone to the Midwest router, back onto the NSFNET backbone to the East Coast router, and finally across the

ARPANET backbone to host *2*. Such a route may or may not be advisable, depending on the policies for network use and the capacity of various routers and backbones.

For most peer backbone configurations, traffic between a pair of geographically close hosts should take a shortest path, independent of the routes chosen for cross-country traffic. For example, traffic from host *3* to host *1* should flow through the West Coast router because it minimizes distance on both backbones.

All these statements sound simple enough, but they are complex to implement for two reasons. First, although the standard IP routing algorithm uses the network portion of an IP address to choose a route, optimal routing in a peer backbone architecture requires individual routes for individual hosts. For our example above, the routing table in host *3* needs different routes for host *1* and host *2*, even though both hosts *1* and *2* attach to the ARPANET backbone. Second, managers of the two backbones must agree to keep routes consistent among all routers or *routing loops* can develop (a routing loop occurs when routes in a set of routers point in a circle).

It is important to distinguish network topology from routing architecture. It is possible, for example, to have a single core system that spans multiple backbone networks. The core machines can be programmed to hide the underlying architectural details and to compute shortest routes among themselves. It is not possible, however, to partition the core system into subsets that each keep partial information without losing functionality. Figure 14.5 illustrates the problem.

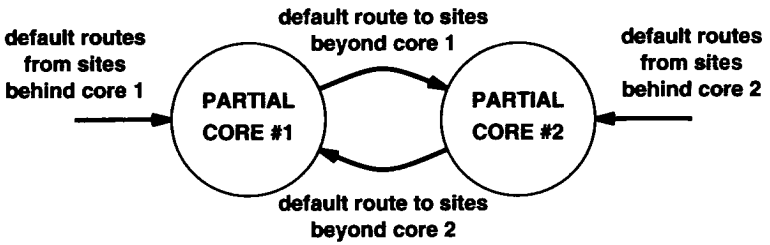

Figure 14.5 An attempt to partition a core routing architecture into two sets of routers that keep partial information and use default routes. Such an architecture results in a routing loop for datagrams that have an illegal (nonexistent) destination.

As the figure shows, outlying routers have default routes to one side of the partitioned core. Each side of the partition has information about destinations on its side of the world and a default route for information on the other side of the world. In such an architecture, any datagram sent to an illegal address will cycle between the two partitions in a routing loop until its time to live counter reaches zero.

We can summarize as follows:

A core routing architecture assumes a centralized set of routers serves as the repository of information about all possible destinations in an internet. Core systems work best for internets that have a single, centrally managed backbone. Expanding the topology to multiple backbones makes routing complex; attempting to partition the core architecture so that all routers use default routes introduces potential routing loops.

14.7 Automatic Route Propagation

We said that the original Internet core system avoided default routes because it propagated complete information about all possible destinations to every core router. Many corporate internets now use a similar scheme — routers in the corporation run programs that communicate routing information. The next sections discuss two basic types of algorithms that compute and propagate routing information, and use the original core routing protocol to illustrate one of the algorithms. A later section describes a protocol that uses the other type of algorithm.

It may seem that automatic route propagation mechanisms are not needed, especially on small internets. However, internets are not static. Connections fail and are later replaced. Networks can become overloaded at one moment and underutilized at the next. The purpose of routing propagation mechanisms is not merely to find a set of routes, but to continually update the information. Humans simply cannot respond to changes fast enough; computer programs must be used. Thus, when we think about route propagation, it is important to consider the dynamic behavior of protocols and algorithms.

14.8 Distance Vector (Bellman-Ford) Routing

The term *distance-vector*† refers to a class of algorithms routers use to propagate routing information. The idea behind distance-vector algorithms is quite simple. The router keeps a list of all known routes in a table. When it boots, a router initializes its routing table to contain an entry for each directly connected network. Each entry in the table identifies a destination network and gives the distance to that network, usually measured in hops (which will be defined more precisely later). For example, Figure 14.6 shows the initial contents of the table on a router that attaches to two networks.

†The terms *vector-distance*, *Ford-Fulkerson*, *Bellman-Ford*, and *Bellman* are synonymous with *distance-vector*; the last two are taken from the names of researchers who published the idea.

Destination	Distance	Route
Net 1	0	direct
Net 2	0	direct

Figure 14.6 An initial distance-vector routing table with an entry for each directly connected network. Each entry contains the IP address of a network and an integer distance to that network.

Periodically, each router sends a copy of its routing table to any other router it can reach directly. When a report arrives at router *K* from router *J*, *K* examines the set of destinations reported and the distance to each. If *J* knows a shorter way to reach a destination, or if *J* lists a destination that *K* does not have in its table, or if *K* currently routes to a destination through *J* and *J*'s distance to that destination changes, *K* replaces its table entry. For example, Figure 14.7 shows an existing table in a router, *K*, and an update message from another router, *J*.

Destination	Distance	Route
Net 1	0	direct
Net 2	0	direct
Net 4	8	Router L
Net 17	5	Router M
Net 24	6	Router J
Net 30	2	Router Q
Net 42	2	Router J

(a)

	Destination	Distance
	Net 1	2
→	Net 4	3
	Net 17	6
→	Net 21	4
	Net 24	5
	Net 30	10
→	Net 42	3

(b)

Figure 14.7 (a) An existing route table for a router *K*, and (b) an incoming routing update message from router *J*. The marked entries will be used to update existing entries or add new entries to *K*'s table.

Note that if *J* reports distance *N*, an updated entry in *K* will have distance *N+1* (the distance to reach the destination from *J* plus the distance to reach *J*). Of course, the routing table entries contain a third column that specifies a next hop. The next hop entry in each initial route is marked *direct delivery*. When router *K* adds or updates an entry in response to a message from router *J*, it assigns router *J* as the next hop for that entry.

The term *distance-vector* comes from the information sent in the periodic messages. A message contains a list of pairs *(V, D)*, where *V* identifies a destination (called the *vector*), and *D* is the distance to that destination. Note that distance-vector algorithms report routes in the first person (i.e., we think of a router advertising, "I can

reach destination V at distance D''). In such a design, all routers must participate in the distance-vector exchange for the routes to be efficient and consistent.

Although distance-vector algorithms are easy to implement, they have disadvantages. In a completely static environment, distance-vector algorithms propagate routes to all destinations. When routes change rapidly, however, the computations may not stabilize. When a route changes (i.e, a new connection appears or an old one fails), the information propagates slowly from one router to another. Meanwhile, some routers may have incorrect routing information.

For now, we will examine a simple protocol that uses the distance-vector algorithm without discussing all the shortcomings. Chapter 16 completes the discussion by showing another distance-vector protocol, the problems that can arise, and the heuristics used to solve the most serious of them.

14.9 Gateway-To-Gateway Protocol (GGP)

The original core routers used a distance-vector protocol known as the *Gateway-to-Gateway Protocol† (GGP)* to exchange routing information. Although GGP only handled classful routes and is no longer part of the TCP/IP standards‡, it does provide a concrete example of distance-vector routing. GGP was designed to travel in IP datagrams similar to UDP datagrams or TCP segments. Each GGP message has a fixed format header that identifies the message type and the format of the remaining fields. Because only core routers participated in GGP, and because core routers were controlled by a central authority, other routers could not interfere with the exchange.

The original core system was arranged to permit new core routers to be added without modifying existing routers. When a new router was added to the core system, it was assigned one or more core *neighbors* with which it communicated. The neighbors, members of the core, already propagated routing information among themselves. Thus, the new router only needed to inform its neighbors about networks it could reach; they updated their routing tables and propagated this new information further.

GGP is a true distance-vector protocol. The information routers exchange with GGP consists of a set of pairs, (N, D), where N is an IP network address, and D is a distance measured in *hops*. We say that a router using GGP *advertises* the networks it can reach and its cost for reaching them.

GGP measures distance in *router hops*, where a router is defined to be zero hops from directly connected networks, one hop from networks that are reachable through one other router, and so on. Thus, the *number of hops* or the *hop count* along a path from a given source to a given destination refers to the number of routers that a datagram encounters along that path. It should be obvious that using hop counts to calculate shortest paths does not always produce desirable results. For example, a path with hop count *3* that crosses three LANs may be substantially faster than a path with hop count *2* that crosses two slow speed serial lines. Many routers use artificially high hop counts for routes across slow networks.

---

†Recall that although vendors adopted the term *IP router*, scientists originally used the term *IP gateway*.

‡The IETF has declared GGP *historic*, which means that it is no longer recommended for use with TCP/IP.

14.10 Distance Factoring

Like most routing protocols, GGP uses multiple message *types*, each with its own format and purpose. A field in the message header contains a code that identifies the specific message type; a receiver uses the code to decide how to process the message. For example, before two routers can exchange routing information, they must establish communication, and some message types are used for that purpose. The most fundamental message type in GGP is also fundamental to any distance-vector protocol: a routing update which is used to exchange routing information.

Conceptually, a routing update contains a list of pairs, where each entry contains an IP network address and the distance to that network. In practice, however, many routing protocols rearrange the information to keep messages small. In particular, observe that few architectures consist of a linear arrangement of networks and routers. Instead, most are hierarchical, with multiple routers attached to each network. Consequently, most distance values in an update are small numbers, and the same values tend to be repeated frequently. To reduce message size, routing protocols often use a technique that was pioneered in GGP. Known as *distance factoring*, the technique avoids sending copies of the same distance number. Instead, the list of pairs is sorted by distance, each distance value is represented once, and the networks reachable at that distance follow. The next chapter shows how other routing protocols factor information.

14.11 Reliability And Routing Protocols

Most routing protocols use connectionless transport. For example, GGP encapsulates messages directly in IP datagrams; modern routing protocols usually encapsulate in UDP†. Both IP and UDP offer the same semantics: messages can be lost, delayed, duplicated, corrupted, or delivered out of order. Thus, a routing protocol that uses them must compensate for failures.

Routing protocols use several techniques to handle delivery problems. Checksums are used to handle corruption. Loss is either handled by *soft state*‡ or through acknowledgement and retransmission. For example, GGP uses an extended acknowledgement scheme in which a receiver can return either a positive or negative acknowledgement.

To handle delivery out of order and the corresponding reply that occurs when an old message arrives, routing protocols often used *sequence numbers*. In GGP, for example, each side chooses an initial number to use for sequencing when communication begins. The other side must then acknowledge the sequence number. After the initial exchange, each message contains the next number in the sequence, which allows the receiver to know whether the message arrived in order. In a later chapter, we will see an example of a routing protocol that uses soft state information.

---

†There are exceptions — the next chapter discusses a protocol that uses TCP.

‡Recall that soft state relies on timeouts to remove old information rather than waiting for a message from the source.

14.12 Link-State (SPF) Routing

The main disadvantage of the distance-vector algorithm is that it does not scale well. Besides the problem of slow response to change mentioned earlier, the algorithm requires the exchange of large messages. Because each routing update contains an entry for every possible network, message size is proportional to the total number of networks in an internet. Furthermore, because a distance-vector protocol requires every router to participate, the volume of information exchanged can be enormous.

The primary alternative to distance-vector algorithms is a class of algorithms known as *link state*, *link status*, or *Shortest Path First*† (*SPF*). The SPF algorithm requires each participating router to have complete topology information. The easiest way to think of the topology information is to imagine that every router has a map that shows all other routers and the networks to which they connect. In abstract terms, the routers correspond to nodes in a graph and networks that connect routers correspond to edges. There is an edge (link) between two nodes if and only if the corresponding routers can communicate directly.

Instead of sending messages that contain lists of destinations, a router participating in an SPF algorithm performs two tasks. First, it actively tests the status of all neighbor routers. In terms of the graph, two routers are neighbors if they share a link; in network terms, two neighbors connect to a common network. Second, it periodically propagates the link status information to all other routers.

To test the status of a directly connected neighbor, a router periodically exchanges short messages that ask whether the neighbor is alive and reachable. If the neighbor replies, the link between them is said to be *up*. Otherwise, the link is said to be *down*‡. To inform all other routers, each router periodically broadcasts a message that lists the status (state) of each of its links. A status message does not specify routes — it simply reports whether communication is possible between pairs of routers. Protocol software in the routers arranges to deliver a copy of each link status message to all participating routers (if the underlying networks do not support broadcast, delivery is done by forwarding individual copies of the message point-to-point).

Whenever a link status message arrives, a router uses the information to update its map of the internet, by marking links up or down. Whenever link status changes, the router recomputes routes by applying the well-known *Dijkstra shortest path algorithm* to the resulting graph. Dijkstra's algorithm computes the shortest paths to all destinations from a single source.

One of the chief advantages of SPF algorithms is that each router computes routes independently using the same original status data; they do not depend on the computation of intermediate machines. Because link status messages propagate unchanged, it is easy to debug problems. Because routers perform the route computation locally, it is guaranteed to converge. Finally, because link status messages only carry information about the direct connections from a single router, the size does not depend on the number of networks in the internet. Thus, SPF algorithms scale better than distance-vector algorithms.

---

†The name "shortest path first" is a misnomer because all routing algorithms seek shortest paths.

‡In practice, to prevent oscillations between the up and down states, most protocols use a *k-out-of-n rule* to test liveness, meaning that the link remains up until a significant percentage of requests have no reply, and then it remains down until a significant percentage of messages receive a reply.

14.13 Summary

To ensure that all networks remain reachable with high reliability, an internet must provide globally consistent routing. Hosts and most routers contain only partial routing information; they depend on default routes to send datagrams to distant destinations. Originally, the global Internet solved the routing problem by using a core router architecture in which a set of core routers each contained complete information about all networks. Routers in the original Internet core system exchanged routing information periodically, meaning that once a single core router learned about a route, all core routers learned about it. To prevent routing loops, core routers were forbidden from using default routes.

A single, centrally managed core system works well for an internet architecture built on a single backbone network. However, a core architecture does not suffice for an internet that consists of a set of separately managed peer backbones that interconnect at multiple places.

When routers exchange routing information they use one of two basic algorithms, distance-vector or SPF. A distance-vector protocol, GGP, was originally used to propagate routing update information throughout the Internet core.

The chief disadvantage of distance-vector algorithms is that they perform a distributed shortest path computation that may not converge if the status of network connections changes continually. Another disadvantage is that routing update messages grow large as the number of networks increases.

The use of SPF routing predates the Internet. One of the earliest examples of an SPF protocol comes from the ARPANET, which used a routing protocol internally to establish and maintain routes among packet switches. The ARPANET algorithm was used for a decade.

FOR FURTHER STUDY

The definition of the core router system and GGP protocol in this chapter comes from Hinden and Sheltzer [RFC 823]. Braden and Postel [RFC 1812] contains further specifications for Internet routers. Almquist [RFC 1716] summarizes later discussions. Braun [RFC 1093] and Rekhter [RFC 1092] discuss routing in the NSFNET backbone. Clark [RFC 1102] and Braun [RFC 1104] both discuss policy-based routing. The next two chapters present protocols used for propagating routing information between separate sites and within a single site.

EXERCISES

14.1 Suppose a router discovers it is about to route an IP datagram back over the same network interface on which the datagram arrived. What should it do? Why?

14.2 After reading RFC 823 and RFC 1812, explain what an Internet core router (i.e., one with complete routing information) should do in the situation described in the previous question.

14.3 How can routers in a core system use default routes to send all illegal datagrams to a specific machine?

14.4 Imagine students experimenting with a router that attaches a local area network to an internet that has a core routing system. The students want to advertise their network to a core router, but if they accidentally advertise zero length routes to arbitrary networks, traffic from the internet will be diverted to their router incorrectly. How can a core protect itself from illegal data while still accepting updates from such "untrusted" routers?

14.5 Which ICMP messages does a router generate?

14.6 Assume a router is using unreliable transport for delivery. How can the router determine whether a designated neighbor is "up" or "down"? (Hint: consult RFC 823 to find out how the original core system solved the problem.)

14.7 Suppose two routers each advertise the same cost, k, to reach a given network, N. Describe the circumstances under which routing through one of them may take fewer total hops than routing through the other one.

14.8 How does a router know whether an incoming datagram carries a GGP message? An OSPF message?

14.9 Consider the distance-vector update shown in Figure 14.7 carefully. For each item updated in the table, give the reason why the router will perform the update.

14.10 Consider the use of sequence numbers to ensure that two routers do not become confused when datagrams are duplicated, delayed, or delivered out of order. How should initial sequence numbers be selected? Why?